

Mastering Java: From Zero to Hero

- By Pluto Academy



Contents

1. Introduction to Programming & Java

- **What is Programming?**
 - **What is Coding?**
 - **Types of Programming Languages**
 - **Why Learn Java?**
 - **History and Evolution of Java**
 - **Features of Java**
 - **Setting Up Java Environment**
 - **Installing JDK & IDE (IntelliJ, Eclipse, VS Code)**
 - **Writing & Running First Java Program**
-

2. Basics of Java

- **Java Syntax & Structure**
- **Variables and Data Types**
- **Operators in Java**
- **Type Casting**
- **Input and Output in Java**

- **Comments in Java**
-

3. Control Flow Statements

- **Conditional Statements**
 - **if-else**
 - **switch-case**
 - **Loops in Java**
 - **for loop**
 - **while loop**
 - **do-while loop**
 - **Break & Continue Statements**
 - **Enhancing Code Readability with Proper Indentation**
-

4. Functions & Methods

- **Defining and Calling Methods**
- **Method Overloading**
- **Static vs Non-Static Methods**
- **Pass by Value in Java**
- **Recursion in Java**

- **Varargs (Variable Arguments)**
-

5. Object-Oriented Programming (OOP)

- **Understanding OOP Concepts**
 - **Classes and Objects**
 - **Constructors in Java**
 - **Encapsulation**
 - **Inheritance**
 - **Polymorphism (Method Overloading & Overriding)**
 - **Abstract Classes & Interfaces**
 - **this & super Keywords**
 - **Static Keyword in Java**
-

6. Arrays and Strings

- **Introduction to Arrays**
- **1D and 2D Arrays**
- **Array Operations (Sorting, Searching, Copying)**
- **String Class & Methods**

- **Array inbuilt Methods**
 - **String inbuilt Methods**
-

7. Exception Handling

- **Types of Errors in Java**
 - **try, catch, finally Blocks**
 - **Multiple Catch Blocks**
 - **throw and throws Keyword**
 - **Custom Exceptions in Java**
-

8. Java Collections Framework (JCF)

- **Introduction to Collections**
 - **List, Set, Map Interfaces**
 - **ArrayList vs LinkedList**
 - **HashSet vs TreeSet**
 - **HashMap vs TreeMap**
-

9. Multithreading & Concurrency

- **Introduction to Threads**
 - **Creating Threads (Thread Class vs Runnable Interface)**
 - **Thread Lifecycle**
 - **Synchronization in Java**
-

10. Future of Java & Career Path

- **Java Trends & Future Scope**
 - **Industries Using Java**
 - **Java Developer Career Roadmap**
 - **Certifications for Java Developers**
 - **Resources & Best Practices to Master Java**
-

Final Section: MCQs & Interview Questions

100 MCQs for Java Beginners to Advanced

- **20 MCQs on Java Basics**
- **20 MCQs on OOP Concepts**
- **20 MCQs on Exception Handling & File Handling**
- **20 MCQs on Multithreading & Collections**
- **20 MCQs on Advanced Java Topics**

100 Java Interview Questions

- **30 Basic Java Interview Questions**
- **30 OOP & Data Structures Interview Questions**
- **20 Advanced Java & Multithreading Questions**
- **20 Problem-Solving & Logical Thinking Questions**

Pluto Academy

Chapter 1. Introduction to Programming

1.1) What is Programming ?

Programming is the process of writing instructions that a computer can understand and execute to perform a specific task.

These instructions are written using a **programming language (Java, C++, C, Python)**.

In simple terms, **programming is like giving a recipe to a computer** to tell it what steps to follow to complete a task. Just as a recipe has ingredients and steps, a program has **data and instructions**.

Example: Imagine programming like **giving directions** to a robot. If you tell the robot:

👉 "Move forward 10 steps, turn left, pick up the ball, and place it on the table."

The robot will follow the instructions **step by step** to complete the task. Similarly, in programming, we write **step-by-step instructions** for a computer to process and execute.

Key Terms in Programming:

- **Code:** The set of instructions written in a programming language.
- **Program:** A collection of code that performs a specific task.
- **Algorithm:** A sequence of logical steps to solve a problem.
- **Debugging:** The process of finding and fixing errors in a program.

1.2) What is Coding ?

Coding is the act of writing instructions in a programming language to build software, websites, applications, or even games.

💡 Think of coding like writing a recipe:

- Ingredients = Data & Variables
- Steps = Instructions & Algorithms
- Cooking = The computer processes and runs your code

While programming refers to the entire process of designing and building software, coding is specifically the writing part of it.

1.3) Types of Programming Languages

Programming languages are **divided into different categories** based on their use and level of abstraction.

1. Low-Level Languages (Close to Machine Language)

- **Machine Language (Binary Code - 0s and 1s)**
- **Assembly Language (Uses symbols & commands like MOV, ADD)**
✦ *Used for system-level programming (e.g., OS Designing, microcontrollers).*

2. High-Level Languages (Easier for Humans to Read & Write)

- **Procedural Languages:** C, Fortran
- **Object-Oriented Languages:** Java, C++, Python
- **Functional Languages:** Lisp, Haskell
✦ *Used for applications, software development, and web development.*

3. Scripting Languages (For Automation & Web Development)

- **Client-side Scripting:** JavaScript, Typescript
- **Server-side Scripting:** Python, PHP, Ruby
✦ *Used for making dynamic websites and automating tasks.*

4. Markup Languages (Not for Programming, But for Structuring Data)

- **HTML** (HyperText Markup Language)
- **XML** (eXtensible Markup Language)

💡 Why So Many Languages?

Each language is **designed for specific tasks**.

For example:

- ✓ **C** is great for **system programming**
- ✓ **Javascript** is great for **Website Development**
- ✓ **Python** is great for **AI & Data Science**
- ✓ **Java** is great for **Enterprise Applications & Android Development**

1.4) Why Learn Java ?

- ☑ **Java is Beginner-Friendly**

Java has a simple and readable syntax. If you have a basic understanding of English, Java code will be easy to follow.

- ☑ **Java is Platform-Independent ("Write Once, Run Anywhere")**

Unlike C or C++, Java does not depend on the operating system. Java programs run on the JVM (Java Virtual Machine), making them work on Windows, macOS, Linux, and even Android!

☒ **Java is Object-Oriented**

Java follows the OOP (Object-Oriented Programming) model, which makes it:

- ✓ More structured
- ✓ Easier to maintain
- ✓ Reusable (code can be reused in different programs)

☒ **Java is Secure & Robust**

Java has built-in security features and automatic memory management (Garbage Collection), making it less prone to crashes & errors.

☒ **Java is Versatile**

You can use Java for:

- Web Development (Spring Boot)
- Mobile App Development (Android)
- Game Development (e.g Minecraft is made in Java !)
- Enterprise Applications (Banking & Business Apps)

☒ **Java is in High Demand**

Java is one of the top programming languages in the job market. Big companies like Google, Amazon, Netflix, and Facebook use Java for various applications.

1.5) History and Evolution of Java

Introduction

Java is one of the most widely used programming languages, known for its portability, security, and robustness. It was created by James Gosling and his team at Sun Microsystems in 1991, initially named "**Oak**" after an oak tree outside Gosling's office. However, due to trademark issues, the name was changed to "**Java**" in 1995.

Early Development (1991–1995)

The goal behind Java's creation was to develop a platform-independent language that could run on various devices, particularly for interactive television. Although the initial idea didn't gain traction in the television industry, Java found its true potential in internet-based applications.

In **1995**, Sun Microsystems officially launched **Java 1.0**, promoting its "**Write Once, Run Anywhere (WORA)**" philosophy. This meant that Java programs could run on any system with a Java Virtual Machine (JVM), making it a game-changer for software development.

Evolution of Java: Major Milestones

Over the years, Java has undergone several upgrades, adding modern features and improving performance, security, and developer productivity.

Java Timeline & Key Features

📌 **Java 1.0 (1995) - Official Launch**

- Introduced core Java APIs, Applets, and the first JVM.
- Laid the foundation for cross-platform compatibility.

🔗 Java 2 (1998) - The Big Upgrade

- Introduced **Swing** for GUI development.
- Added the **Collections Framework**, providing a unified architecture for working with sets, lists, and maps.

🔗 Java 5 (2004) - Modernization with Generics

- Introduced **Generics**, improving type safety and reusability.
- Added **AutoBoxing/Unboxing**, reducing manual conversions between primitives and wrapper classes.
- Introduced the **Enhanced for-loop** and **Annotations**.

🔗 Java 8 (2014) - The Functional Revolution

- Introduced **Lambda Expressions**, enabling functional programming.
- Added the **Streams API**, simplifying data manipulation and parallel processing.
- Introduced the **Date and Time API**, replacing the outdated `java.util.Date`.

🔗 Java 9 – Java 21 (2017–Present) - Continuous Improvements

- **Java 9 (2017)**: Introduced the **Module System** (Project Jigsaw) for better scalability.
- **Java 10 (2018)**: Introduced **Local-Variable Type Inference (var)**.
- **Java 11 (2018)**: Introduced **Long-Term Support (LTS)** model and removed JavaFX.
- **Java 17 (2021)**: Another **LTS release**, improved performance and security.
- **Java 21 (2023, Latest LTS)**: Enhanced Virtual Threads, Pattern Matching, and Record Patterns.

Java's Future

Java continues to evolve, with new features being released every six months. The focus remains on improving **performance, security, cloud-native capabilities, and developer experience**. With its strong ecosystem, Java is expected to remain a dominant language for years to come.

1.6) Features of Java

Java is famous for its **powerful features** that make it reliable and widely used.

💧 1. Platform Independent

Java's **"Write Once, Run Anywhere"** approach allows it to run on any OS.

💧 2. Object-Oriented Programming (OOP)

Java follows **OOP principles**, making coding more modular, scalable, and reusable.

💧 3. Automatic Memory Management

The **Garbage Collector (GC)** removes unused objects from memory **automatically**.

💧 4. Multi-threading Support

Java allows **multiple tasks to run simultaneously** using **threads**, improving performance.

💧 5. Security

Java has **built-in security features**, such as:

- ✓ No direct memory access (unlike C/C++)
- ✓ Strong exception handling
- ✓ Secure class loading

💧 6. Open-Source & Large Community Support

Java has **free tools & libraries**, plus a **huge developer community** for support.

1.7) Setting Up Java Environment

Introduction

To start coding in Java, you need to set up your development environment properly. This involves installing the **Java Development Kit (JDK)**, setting up system environment variables, choosing an **Integrated Development Environment (IDE)** like **VsCode** or **IntelliJ Idea**, and writing your first Java program. Let's go through each step in detail.

Step 1: Install JDK (Java Development Kit)

The JDK is a software development kit that provides everything needed to compile and run Java applications. It includes:

- ✓ **Java Compiler (javac)** – Translates Java source code into bytecode.
- ✓ **Java Runtime Environment (java)** – Runs compiled Java applications.
- ✓ **Development Tools** – Includes a debugger, JAR utility, and documentation generator.

◇ Download & Install JDK

1 Go to Oracle's official site:

- Open your web browser and go to the official Oracle website:
<https://www.oracle.com/java/technologies/javase-downloads.html>
- Select the latest version of **JDK (Java SE Development Kit)** for your operating system (Windows, macOS, or Linux).

2 Download the JDK Installer:

- Choose the appropriate installer based on your OS:
 - **Windows:** .exe file
 - **macOS:** .dmg file
 - **Linux:** .tar.gz or .rpm file
- Click on the download button and accept the terms and conditions.

3 Install the JDK:

- **Windows:**

- Double-click the `.exe` file and follow the on-screen instructions.
- By default, it installs to `C:\Program Files\Java\jdk-XX`.
- **macOS:**
 - Open the `.dmg` file and drag the JDK folder into the `/Library/Java/JavaVirtualMachines/` directory.
- **Linux:**
 - Extract the `.tar.gz` file using the command: `tar -xvzf jdk-XX.tar.gz`.
 - Move it to `/usr/lib/jvm/` or another preferred directory.

Note: Further Step is only for Windows

◇ Set Up Environment Variables (Windows)

After installing the JDK, you need to configure **PATH** and **JAVA_HOME** environment variables to use Java from the command line.

1 Open Environment Variables:

- Press `Win + R`, type `sysdm.cpl`, and hit **Enter**.
- Go to the **Advanced** tab → Click **Environment Variables**.

2 Set JAVA_HOME:

- Click **New** under **System Variables**.
- Set **Variable Name** to `JAVA_HOME`.
- Set **Variable Value** to the installation path, e.g., `C:\Program Files\Java\jdk-XX`.

Note: XX → Indicate the current version of JDK you Installed.

3 Update the PATH Variable:

- Find `Path` under **System Variables**, select it, and click **Edit**.
- Click **New** and add `%JAVA_HOME%\bin`.
- Click **OK** to save changes.

4 Verify Installation:

- Open **Command Prompt** (`cmd`) and type:

```
java -version
```

- If installed correctly, you should see the Java version displayed.
-

Step 2: Install an IDE (Integrated Development Environment)

An IDE is a software application that simplifies coding by providing features like syntax highlighting, auto-completion, debugging, and project management.

☒ Best IDEs for Java

- ✓ **IntelliJ IDEA** – Recommended for beginners due to its smart code assistance and robust debugging tools.
- ✓ **Eclipse** – Suitable for large projects and enterprise applications.
- ✓ **VS Code** – Lightweight, requires Java extension for full support.

◇ Installing IntelliJ IDEA (Recommended)

- 1 Go to <https://www.jetbrains.com/idea/download/>.
 - 2 **Download** the Community Edition (free version).
 - 3 **Install** it by following the setup instructions.
 - 4 **Launch IntelliJ IDEA**, and select **Create New Project**.
 - 5 **Choose Java SDK** (if prompted, select the installed JDK directory).
-

Step 3: Write & Run Your First Java Program

Now that your environment is set up, let's write and execute a simple Java program.

◊ Writing Your First Java Program

- 1 Open IntelliJ IDEA or any text editor.
- 2 Create a new file and name it `HelloWorld.java`.
- 3 Copy and paste the following code:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

◊ Running Java Code

Using IntelliJ IDEA:

- 1 Click on **Run** → **Run 'HelloWorld'**.
- 2 The output will be displayed in the console.

Using Command Line:

- 1 Open **Command Prompt** (`cmd`) or Terminal.
- 2 Navigate to the directory where the file is saved.
- 3 Compile the program:

```
javac HelloWorld.java
```

- 4 Run the compiled file:

```
java HelloWorld
```

5 Output:

```
Hello, World!
```

Conclusion

By following these steps, you have successfully set up Java on your system. Now you understand:

- ✓ What programming is
- ✓ Why Java is powerful
- ✓ Java's history, features & setup

Note: The next chapter will help you understand the syntax and structure of the above code.

Chapter 2: Basics of Java

In this chapter, we will explore the fundamentals of Java, including its syntax, structure, variables, data types, operators, type casting, input/output, and comments. By the end, you will have a solid understanding of how Java programs are structured and how to perform basic operations.

2.1) Java Syntax & Structure

Every Java program follows a specific structure. Let's break it down using the classic "Hello, World!" program.

Hello World Program in Java

```
// This is a simple Java program
public class HelloWorld { // Class Declaration
    public static void main(String[] args) { // Main Method
        System.out.println("Hello, World!"); // Print Statement
    }
}
```

Breaking Down the Structure:

1 Class Declaration (`public class HelloWorld`)

- Every Java program must have a class.
- The class name **must match** the filename (e.g., `HelloWorld.java`).
- `public` means it can be accessed from anywhere.

2 Main Method (`public static void main(String[] args)`)

- This is where Java **starts execution**.
- `public` → Accessible from anywhere.
- `static` → No need to create an object of the class to run it.
- `void` → This method does not return any value.
- `String[] args` → Accepts **command-line arguments**.

3 Print Statement (`System.out.println("Hello, World!");`)

- `System.out.println()` is used to print output to the console.
- The text inside **double quotes** (" ") will be displayed on the screen.

Key Points About Java Syntax:

- ✓ Java is **case-sensitive** (e.g., **Main** and **main** are different).
 - ✓ Every **statement must end** with a semicolon (;).
 - ✓ Java files **must be saved with** a **.java** extension.
 - ✓ Java execution **starts from the main** method.
-

2.2) Variables and Data Types

What is a Variable?

A **variable** is a container used to store data in memory.

✦ Think of a variable as a **storage box** where you keep a value.

Declaring a Variable in Java

```
dataType variableName = value;
```

Example:

```
int age = 25; // "age" stores the number 25
```

Types of Variables in Java:

- 1 Local Variable** – Declared **inside a method** and only accessible within it.
- 2 Instance Variable** – Declared **inside a class** but **outside any method**.
- 3 Static Variable** – Declared using **static** and **shared by all objects**.

Note: Both Image refers the same Class Example

```
class Example {  
    // Instance variable (belongs to an object)  
    int instanceVar = 10;  
  
    // Static variable (shared among all objects)  
    static int staticVar = 20;  
  
    void method() {  
        // Local variable (only accessible inside this method)  
        int localVar = 30;  
  
        System.out.println("Local Variable: " + localVar);  
        System.out.println("Instance Variable: " + instanceVar);  
        System.out.println("Static Variable: " + staticVar);  
    }  
}
```

```
public static void main(String[] args) {  
    Example obj1 = new Example();  
    Example obj2 = new Example();  
  
    obj1.method();  
    obj2.instanceVar = 50; // Changing instance variable of obj2  
    Example.staticVar = 100; // Changing static variable (affects all instances)  
  
    System.out.println("\nAfter Modification:");  
    obj1.method(); // Instance variable remains unchanged for obj1  
    obj2.method(); // Instance variable changed for obj2, static variable changed for all  
}  
}
```

Output:

```
Local Variable: 30
Instance Variable: 10
Static Variable: 20
```

```
After Modification:
Local Variable: 30
Instance Variable: 10
Static Variable: 100
```

```
Local Variable: 30
Instance Variable: 50
Static Variable: 100
```

Java Data Types:

Java has **two types** of data types:

- ✦ 1. **Primitive Data Types** (stores simple values)
- ✦ 2. **Non-Primitive Data Types** (stores complex data like arrays & objects)

1. Primitive Data Types in Java

Data Type	Size	Example Values
byte	1 byte	-128 to 127
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2 billion to 2 billion
long	8 bytes	Large numbers (e.g., 123456789L)
float	4 bytes	Decimal numbers (e.g., 3.14f)
double	8 bytes	More precise decimal (e.g., 3.14159265)
char	2 bytes	Single character (e.g., 'A')
boolean	1 bit	true or false

2. Non-Primitive Data Types in Java

📌 Examples:

- **String** (stores text)
- **Array** (stores multiple values)
- **Class & Object** (OOP concepts)
- **Interfaces** (More on this in OOP section)

Example of Different Data Types in Java

```
public class DataTypesExample {  
    public static void main(String[] args) {  
        int age = 25; // Integer  
        double pi = 3.14159; // Double  
        char grade = 'A'; // Character  
        boolean isJavaFun = true; // Boolean  
        String message = "Hello, Java!"; // String  
  
        System.out.println(age);  
        System.out.println(pi);  
        System.out.println(grade);  
        System.out.println(isJavaFun);  
        System.out.println(message);  
    }  
}
```

2.3) Operators in Java

Operators are symbols that perform operations on variables and values.

Types of Operators in Java:

1 Arithmetic Operators (+, -, *, /, %)

```
public class ArithmeticOperators {  
    public static void main(String[] args) {  
        int a = 10, b = 5;  
  
        System.out.println("Arithmetic Operators:");  
        System.out.println("Addition: " + (a + b));    // 10 + 5 = 15  
        System.out.println("Subtraction: " + (a - b)); // 10 - 5 = 5  
        System.out.println("Multiplication: " + (a * b)); // 10 * 5 = 50  
        System.out.println("Division: " + (a / b));    // 10 / 5 = 2  
        System.out.println("Modulus: " + (a % b));    // 10 % 5 = 0  
    }  
}
```

Output:

```
Arithmetic Operators:  
Addition: 15  
Subtraction: 5  
Multiplication: 50  
Division: 2  
Modulus: 0
```

2 Relational (Comparison) Operators (==, !=, >, <, >=, <=)

```
public class RelationalOperators {  
    public static void main(String[] args) {  
        int a = 10, b = 5;  
  
        System.out.println("Relational Operators:");  
        System.out.println("a == b: " + (a == b)); // false  
        System.out.println("a != b: " + (a != b)); // true  
        System.out.println("a > b: " + (a > b));    // true  
        System.out.println("a < b: " + (a < b));    // false  
        System.out.println("a >= b: " + (a >= b)); // true  
        System.out.println("a <= b: " + (a <= b)); // false  
    }  
}
```

Output:

```
Relational Operators:
```

```
a == b: false  
a != b: true  
a > b: true  
a < b: false  
a >= b: true  
a <= b: false
```

3 Logical Operators (&&, ||, !)

```
public class LogicalOperators {  
    public static void main(String[] args) {  
        boolean x = true, y = false;  
  
        System.out.println("Logical Operators:");  
        System.out.println("x && y: " + (x && y)); // false  
        System.out.println("x || y: " + (x || y)); // true  
        System.out.println("!x: " + (!x));          // false  
    }  
}
```

Output:

```
Logical Operators:  
x && y: false  
x || y: true  
!x: false
```

4 Bitwise Operators (&, |, ^, ~, <<, >>)

```

public class BitwiseOperators {
    public static void main(String[] args) {
        int a = 5, b = 3; // Binary: a = 0101, b = 0011

        System.out.println("Bitwise Operators:");
        System.out.println("a & b: " + (a & b)); // 0101 & 0011 = 0001 (1)
        System.out.println("a | b: " + (a | b)); // 0101 | 0011 = 0111 (7)
        System.out.println("a ^ b: " + (a ^ b)); // 0101 ^ 0011 = 0110 (6)
        System.out.println("~a: " + (~a)); // ~0101 = 1010 (-6 in 2's complement)
        System.out.println("a << 1: " + (a << 1)); // 0101 << 1 = 1010 (10)
        System.out.println("a >> 1: " + (a >> 1)); // 0101 >> 1 = 0010 (2)
    }
}

```

Output:

```

Bitwise Operators:
a & b: 1
a | b: 7
a ^ b: 6
~a: -6
a << 1: 10
a >> 1: 2

```

5 Assignment Operators (=, +=, -=, *=, /=, %=)

```
public class AssignmentOperators {  
    public static void main(String[] args) {  
        int a = 10;  
  
        System.out.println("Assignment Operators:");  
        System.out.println("a = " + a);  
        a += 5; // a = a + 5  
        System.out.println("a += 5: " + a);  
        a -= 3; // a = a - 3  
        System.out.println("a -= 3: " + a);  
        a *= 2; // a = a * 2  
        System.out.println("a *= 2: " + a);  
        a /= 4; // a = a / 4  
        System.out.println("a /= 4: " + a);  
        a %= 3; // a = a % 3  
        System.out.println("a %= 3: " + a);  
    }  
}
```

Output:

```
Assignment Operators:  
a = 10  
a += 5: 15  
a -= 3: 12  
a *= 2: 24  
a /= 4: 6  
a %= 3: 0
```

6 Increment & Decrement Operators (++ , --)

```

public class IncrementDecrementOperators {
    public static void main(String[] args) {
        int a = 10;

        System.out.println("Increment & Decrement Operators:");
        System.out.println("Initial value of a: " + a);
        System.out.println("Post-increment (a++): " + (a++)); // Uses a, then increments
        System.out.println("After post-increment: " + a);
        System.out.println("Pre-increment (++a): " + (++a)); // Increments, then uses
        System.out.println("Post-decrement (a--): " + (a--)); // Uses a, then decrements
        System.out.println("After post-decrement: " + a);
        System.out.println("Pre-decrement (--a): " + (--a)); // Decrements, then uses
    }
}

```

Output:

```

Increment & Decrement Operators:
Initial value of a: 10
Post-increment (a++): 10
After post-increment: 11
Pre-increment (++a): 12
Post-decrement (a--): 12
After post-decrement: 11
Pre-decrement (--a): 10

```

7 Ternary Operator (condition ? value1 : value2)

```

public class TernaryOperator {
    public static void main(String[] args) {
        int a = 10, b = 20;

        System.out.println("Ternary Operator:");
        int min = (a < b) ? a : b;
        System.out.println("Minimum value: " + min);
    }
}

```

Output:

```
Ternary Operator:  
Minimum value: 10
```

2.4) Type Casting

Type Casting is **converting one data type into another**.

Types of Type Casting in Java

1 Implicit Type Casting (Widening Conversion) – Smaller type → Larger type (automatic).

2 Explicit Type Casting (Narrowing Conversion) – Larger type → Smaller type (manual).

Example of Type Casting in Java:

```
public class TypeCastingExample {  
    public static void main(String[] args) {  
        int num = 10;  
        double decimalNum = num; // int to double (automatic widening)  
        double pi = 3.14;  
        int piInt = (int) pi; // double to int (manual narrowing)  
  
        System.out.println(decimalNum);  
        System.out.println(piInt);  
    }  
}
```

Output:

```
10.0  
3
```

Note: The first conversion is ITC and second conversion is ETC

2.5) Input and Output in Java

In Java, handling input and output (I/O) is a fundamental task, allowing us to interact with users and display results on the console or take input from the user.

Output in Java:

System.out.print() vs System.out.println()

- **System.out.print():** This method prints the given message but does not move to a new line after the output.
- **System.out.println():** This method prints the given message and moves to a new line after the output.

Example:

```
public class OutputExample {  
    public static void main(String[] args) {  
        System.out.print("Hello "); // No new line after this  
        System.out.println("World!"); // New line after this  
    }  
}
```

Output:

```
Hello World!
```

Input in Java:

Using the Scanner Class for Taking Input

To read data entered by the user, Java provides the Scanner class (part of the `java.util` package). It allows us to take input for different data types (e.g., `int`, `String`, `double`, etc.).

Example:

```

import java.util.Scanner; // Import the Scanner class

public class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // Create Scanner object

        // Taking input for String
        System.out.print("Enter your name: ");
        String name = sc.nextLine(); // Read user input as String

        // Taking input for Integer
        System.out.print("Enter your age: ");
        int age = sc.nextInt(); // Read user input as Integer

        // Display the entered information
        System.out.println("Hello " + name + ", you are " + age + " years old.");

        sc.close(); // Close the scanner to prevent resource leak
    }
}

```

Explanation:

- **Scanner sc = new Scanner(System.in);**: Creates a new Scanner object to read input from the console.
- **nextLine()**: Reads a whole line of text input (including spaces).
- **nextInt()**: Reads an integer input.
- **sc.close()**: Closes the Scanner object after usage.

Output:

```
Enter your name: Alice
Enter your age: 25
Hello Alice, you are 25 years old.
```

2.6) Comments in Java

Comments are used to explain and document your code, making it more understandable for others (or yourself). Comments are ignored by the Java compiler and have no impact on the execution of the program.

Types of Comments in Java

1 Single-line Comment (//)

- Used for writing comments that occupy a single line.
- Everything after `//` is treated as a comment and is ignored by the compiler.

Example:

```
// This is a single-line comment

System.out.println("Hello, World!"); // Print message
```

2 Multi-line Comment (/* ... */)

- Used for writing comments that span multiple lines.
- You can comment out a block of code or add long descriptions.

Example:

```
/* This is a multi-line comment

   that spans multiple lines.

   It can be used to describe a large section of code. */

System.out.println("Hello, World!");
```

Key Points:

- Single-line comments: Useful for brief explanations or marking sections of code.
- Multi-line comments: Useful for longer explanations or temporarily disabling a section of code.

Conclusion

Now you understand:

- ✓ Java syntax & structure
- ✓ Variables and data types
- ✓ Operators and type casting
- ✓ Input and Output in Java
- ✓ How to use comments

 **Next, we'll explore Control Flow Statements (if-else, loops, switch-case, etc.)!**

Chapter 3. Control Flow Statements in Java

Introduction:

Control flow statements in Java determine the flow of execution in a program. These statements help in decision-making and repeating tasks based on conditions.

3.1) Conditional Statements

Conditional statements execute different blocks of code based on specified conditions.

(1) if-else Statement

The `if-else` statement executes a block of code conditionally.

Syntax:

```
if (condition) {  
    // Code runs if condition is true  
} else {  
    // Code runs if condition is false  
}
```

Example:

```
public class IfElseExample {  
    public static void main(String[] args) {  
        int age = 18;  
  
        if (age >= 18) {  
            System.out.println("You are eligible to vote.");  
        } else {  
            System.out.println("You are not eligible to vote.");  
        }  
    }  
}
```

Output:

```
You are eligible to vote.
```

(2) if-else-if Ladder

When multiple conditions need to be checked, use `if-else-if`.

Example:

```
public class IfElseIfExample {  
    public static void main(String[] args) {  
        int marks = 85;  
  
        if (marks >= 90) {  
            System.out.println("Grade: A");  
        } else if (marks >= 75) {  
            System.out.println("Grade: B");  
        } else if (marks >= 60) {  
            System.out.println("Grade: C");  
        } else {  
            System.out.println("Grade: F");  
        }  
    }  
}
```

Output:

Grade: B

(3) Nested if Statement

An `if` statement inside another `if` is called a nested `if` statement.

Example:

```
public class NestedIfExample {
    public static void main(String[] args) {
        int age = 20;
        boolean hasVoterID = true;

        if (age >= 18) {
            if (hasVoterID) {
                System.out.println("You can vote.");
            } else {
                System.out.println("You need a voter ID to vote.");
            }
        } else {
            System.out.println("You are not old enough to vote.");
        }
    }
}
```

Output:

```
You can vote.
```

3.2) switch-case Statement

Used when multiple conditions need to be checked efficiently.

Syntax:

```
switch (expression) {
    case value1:
        // Code for case 1
        break;
    case value2:
        // Code for case 2
        break;
    default:
        // Code if no case matches
}
```

Example:

```
public class SwitchExample {  
    public static void main(String[] args) {  
        int day = 3;  
  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  
                break;  
            case 2:  
                System.out.println("Tuesday");  
                break;  
            case 3:  
                System.out.println("Wednesday");  
                break;  
            default:  
                System.out.println("Invalid Day");  
        }  
    }  
}
```

Output:

Wednesday

3.3) Loops in Java

Loops are used to execute a block of code multiple times.

(1) for Loop

The **for** loop is used when the number of iterations is known beforehand.

Syntax:

```
for (initialization; condition; increment/decrement) {  
    // Code to execute in each iteration  
}
```

Example:

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Iteration: " + i);  
        }  
    }  
}
```

Output:

```
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4  
Iteration: 5
```

(2) while Loop

The **while** loop is used when the number of iterations is not known.

Syntax:

```
while (condition) {  
    // Code executes while condition is true  
}
```

Example:

```
public class WhileLoopExample {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println("Iteration: " + i);  
            i++;  
        }  
    }  
}
```

Output:

```
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4  
Iteration: 5
```

(3) do-while Loop

The **do-while** loop is similar to **while**, but it executes at least once.

Syntax:

```
do {  
    // Code executes at least once  
} while (condition);
```

Example:

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.println("Iteration: " + i);  
            i++;  
        } while (i <= 5);  
    }  
}
```

Output:

```
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4  
Iteration: 5
```

3.4) Break & Continue Statements

(1) break Statement

The **break** statement stops the loop immediately.

Example:

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) {  
                break;  
            }  
            System.out.println("Iteration: " + i);  
        }  
    }  
}
```

Output:

```
Iteration: 1  
Iteration: 2
```

(2) continue Statement

The `continue` statement skips the current iteration.

Example:

```
public class ContinueExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) {  
                continue;  
            }  
            System.out.println("Iteration: " + i);  
        }  
    }  
}
```

Output:

```
Iteration: 1  
Iteration: 2  
Iteration: 4  
Iteration: 5
```

Conclusion:

Now you understand:

- ✓ **if-else, switch-case** for decision-making
- ✓ **for, while, do-while** loops
- ✓ **break** and **continue** for controlling loops
- 🚀 **Next, we'll explore Strings and Arrays in Java!**

Chapter 4: Functions & Methods in Java

Introduction to Methods in Java

Methods in Java allow us to break down a complex problem into smaller, reusable blocks of code. A method is a block of statements that performs a specific task.

Why Use Methods?

- ✓ **Code Reusability:** Methods allow us to write code once and use it multiple times.
- ✓ **Readability & Maintainability:** Code becomes more structured and easier to manage.
- ✓ **Modularity:** Breaking down the program into smaller sections makes debugging and testing easier.

4.1) Defining and Calling Methods

A method must be declared inside a class. It has a return type, name, parentheses () (with or without parameters), and a method body enclosed in { }.

Syntax of a Method

```
returnType methodName(parameters) {  
    // Method body  
    return value; // If returnType is not void  
}
```

Example: Creating and Calling a Method

```
public class MethodExample {  
    // Method definition  
    public static void greet() {  
        System.out.println("Hello, welcome to Java!");  
    }  
  
    public static void main(String[] args) {  
        // Method call  
        greet();  
    }  
}
```

Output:

```
Hello, welcome to Java!
```

4.2) Method Overloading

Method overloading allows multiple methods in a class to have the same name but with different parameters (number, type, or both).

Example: Method Overloading

```
public class OverloadingExample {  
    // Method with one parameter  
    public static void display(int num) {  
        System.out.println("Number: " + num);  
    }  
  
    // Method with two parameters  
    public static void display(int num1, int num2) {  
        System.out.println("Numbers: " + num1 + " and " + num2);  
    }  
  
    public static void main(String[] args) {  
        display(5);  
        display(10, 20);  
    }  
}
```

Output:

```
Number: 5  
Numbers: 10 and 20
```

Note: In the above code we have two methods with the same name “display()” but we pass different numbers of parameters that is why we can say that method “display” is overloaded.

4.3) Static vs Non-Static Methods

(1) Static Methods

- Declared using the `static` keyword.
- Can be called without creating an instance of the class.
- Cannot directly access instance variables or non-static methods.

Example: Static Method

```
public class StaticExample {  
    public static void showMessage() {  
        System.out.println("This is a static method.");  
    }  
  
    public static void main(String[] args) {  
        showMessage();  
    }  
}
```

Output:

```
This is a static method.
```

(2) Non-Static Methods

- Require an object of the class to be invoked.
- Can access both static and non-static members of the class.

Example: Non-Static Method

```
public class NonStaticExample {  
    public void displayMessage() {  
        System.out.println("This is a non-static method.");  
    }  
  
    public static void main(String[] args) {  
        NonStaticExample obj = new NonStaticExample();  
        obj.displayMessage();  
    }  
}
```

Output:

```
This is a non-static method.
```

4.4) Pass by Value in Java

In Java, arguments are **passed by value**, meaning a copy of the variable is passed, and changes made inside the method do not affect the original variable.

Example: Pass by Value

```
public class PassByValueExample {  
    public static void modifyValue(int num) {  
        num = num + 10;  
    }  
  
    public static void main(String[] args) {  
        int x = 5;  
        modifyValue(x);  
        System.out.println("Value after method call: " + x);  
    }  
}
```

Output:

```
Value after method call: 5
```

Note: The value of `x` remains unchanged outside the method.

4.5 Recursion in Java

Recursion is a process where a method calls itself to solve smaller subproblems.

Example: Factorial using Recursion

```
public class RecursionExample {  
    public static int factorial(int n) {  
        if (n == 0)  
            return 1;  
        else  
            return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
        int result = factorial(5);  
        System.out.println("Factorial of 5 is: " + result);  
    }  
}
```

Output:

```
Factorial of 5 is: 120
```

🔗 Recursive Function Breakdown:

Factorial is computed as:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

1. $\text{factorial}(5) \rightarrow 5 * \text{factorial}(4)$
2. $\text{factorial}(4) \rightarrow 4 * \text{factorial}(3)$
3. $\text{factorial}(3) \rightarrow 3 * \text{factorial}(2)$
4. $\text{factorial}(2) \rightarrow 2 * \text{factorial}(1)$
5. $\text{factorial}(1) \rightarrow 1 * \text{factorial}(0)$
6. **Base Case:** $\text{factorial}(0) \rightarrow 1$ (Stops recursion)

Returning the value to caller of that function (main method)

Virtual Representation:

```
factorial(5)
→ 5 * factorial(4)
→ 4 * factorial(3)
→ 3 * factorial(2)
→ 2 * factorial(1)
→ 1 * factorial(0)
→ 1 (Base Case)
```

Now, the recursion starts returning:

```
factorial(1) = 1 * 1 = 1
factorial(2) = 2 * 1 = 2
factorial(3) = 3 * 2 = 6
factorial(4) = 4 * 6 = 24
factorial(5) = 5 * 24 = 120
```

4.6) Varargs (Variable Arguments)

Varargs (`...`) allow a method to accept multiple arguments of the same type without defining multiple parameters.

Syntax:

```
returnType methodName(dataType... variableName) {
    // Method body
}
```

Example: Varargs in Java

```

public class VarargsExample {
    public static void sum(int... numbers) {
        int total = 0;
        for (int num : numbers) {
            total += num;
        }
        System.out.println("Sum: " + total);
    }

    public static void main(String[] args) {
        sum(10, 20);
        sum(5, 10, 15);
    }
}

```

Note: Above we used “for each” loop a variation of standard “for” loop and also the point to noted is that we write “int ...numbers” It means that we are saying “numbers” can accept any number of parameter so it internally acts an array.

Output:

```

Sum: 30
Sum: 30

```

Note: The method can take any number of arguments.

Conclusion:

Now you understand:

- ✓ How to define and call methods in Java
- ✓ Method Overloading and its use cases
- ✓ Difference between Static and Non-Static methods

✓ Java's Pass by Value behavior

✓ Recursion with real examples

✓ How to use Varargs for flexible method arguments

🚀 Next, we'll explore **Object-Oriented Programming (OOP)** in Java!

Pluto Academy

Chapter 5. Object-Oriented Programming (OOP) in Java

Object-Oriented Programming (OOP) is a programming paradigm that uses objects to design applications. Java follows the OOP principles, making it powerful, modular, and reusable.

5.1) Understanding OOP Concepts

OOP is based on the following principles:

- ◆ **Encapsulation** – Wrapping data and code together (e.g., medicine in a capsule).
- ◆ **Inheritance** – Deriving new classes from existing ones (e.g., child inheriting traits from parents).
- ◆ **Polymorphism** – Same function behaving differently in different scenarios (e.g., a person acting as a student in school and as a player on a field).
- ◆ **Abstraction** – Hiding implementation details and showing only essential features (e.g., driving a car without knowing how the engine works).

5.2) Classes and Objects in Java

What is a Class ?

A **class** is a blueprint or template for creating objects. It defines:

- **Properties (Attributes/Variables)** → Represent characteristics of an object.
- **Methods (Functions/Behaviors)** → Define actions the object can perform.

Example: Class Creation

```
// Defining a class named Car
class Car {
    // Instance variables (Properties of Car)
    String brand;
    String model;
    int year;

    // Constructor (Special method to initialize an object)
    Car(String brand, String model, int year) {
        this.brand = brand;
        this.model = model;
        this.year = year;
    }

    // Method to display car details
    void displayInfo() {
        System.out.println("Car Brand: " + brand);
        System.out.println("Car Model: " + model);
        System.out.println("Manufacturing Year: " + year);
    }
}
```

Note: To create class we use keyword “class followed by class_name”

What is an Object ?

An **object** is an **instance** of a class, created using the **new** keyword.

Each object has its **own copy** of instance variables but **shares the class methods**.

```
public class Main {  
    public static void main(String[] args) {  
        // Creating objects of Car class  
        Car car1 = new Car("Toyota", "Corolla", 2020);  
        Car car2 = new Car("Honda", "Civic", 2022);  
  
        // Calling the method to display car details  
        System.out.println("Car 1 Details:");  
        car1.displayInfo();  
  
        System.out.println("\nCar 2 Details:");  
        car2.displayInfo();  
    }  
}
```

Output:

```
Car 1 Details:  
Car Brand: Toyota  
Car Model: Corolla  
Manufacturing Year: 2020  
  
Car 2 Details:  
Car Brand: Honda  
Car Model: Civic  
Manufacturing Year: 2022
```

Note: The data is printed because we create an object of class Car

5.3) Constructors in Java

A **constructor** initializes an object when it is created.

Key Points:

- It has the same name as the class.
- It does not return any value (not even **void**).

- It is automatically called when an object is created.

Types of Constructors:

Default Constructor (No Arguments)

A **default constructor** does **not take any parameters**. It initializes instance variables with **default values**.

```
// Class with a Default Constructor
class Person {
    String name;
    int age;

    // Default Constructor
    Person() {
        name = "Unknown";
        age = 18;
    }

    // Method to display details
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object of Person class (default constructor is called)
        Person p1 = new Person();

        // Displaying default values
        p1.display();
    }
}
```

Output:

```
Name: Unknown
Age: 18
```

2 Parameterized Constructor (Takes Parameters)

A **parameterized constructor** takes **arguments** to **assign custom values** to instance variables.

```
// Class with a Parameterized Constructor
class Car {
    String brand;
    int speed;

    // Parameterized Constructor
    Car(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }

    // Method to display car details
    void display() {
        System.out.println("Car Brand: " + brand);
        System.out.println("Max Speed: " + speed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating objects with different values
        Car car1 = new Car("Toyota", 180);
        Car car2 = new Car("BMW", 250);

        // Displaying details
        System.out.println("Car 1 Details:");
        car1.display();

        System.out.println("\nCar 2 Details:");
        car2.display();
    }
}
```

Output:

```
Car 1 Details:
Car Brand: Toyota
Max Speed: 180 km/h

Car 2 Details:
Car Brand: BMW
Max Speed: 250 km/h
```

5.4) Encapsulation

Encapsulation is **data hiding** by making variables **private** and accessing them via **public** methods.

Example:

```
class BankAccount {
    private double balance;

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.deposit(1000);
        System.out.println("Balance: " + account.getBalance());
    }
}
```

Output:

Balance: 1000.0

5.5) Inheritance

Inheritance allows a class (**child**) to inherit properties and methods from another class (**parent**).

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animals make sounds");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output:

Dog barks

5.6) Polymorphism

Polymorphism allows the same method to perform different actions.

Method Overloading (Compile-Time Polymorphism)

Same method name, different parameters.

```
class MathUtils {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MathUtils mu = new MathUtils();  
        System.out.println(mu.add(5, 10));  
        System.out.println(mu.add(5.5, 2.2));  
    }  
}
```

Note: Method overloading is also a type of polymorphism

Output:

```
15  
7.7
```

Method Overriding (Runtime Polymorphism)

A child class modifies a method from its parent class.

```
class Parent {  
    void show() {  
        System.out.println("Parent class");  
    }  
}  
  
class Child extends Parent {  
    void show() {  
        System.out.println("Child class");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Parent obj = new Child();  
        obj.show();  
    }  
}
```

Output:

```
Child class
```

5.7) Abstract Classes & Interfaces

1 Abstract Classes

An **abstract class** in Java is a **class that cannot be instantiated**. It is meant to be **extended** by subclasses and may contain **abstract methods** (methods without a body).

✓ Key Features:

- Can have **both abstract and concrete methods**.

- Used for **partial implementation**.
- **Must be inherited** by subclasses.

Example:

```
// Abstract class
abstract class Animal {
    // Abstract method (must be implemented by subclasses)
    abstract void makeSound();

    // Concrete method (has a body)
    void sleep() {
        System.out.println("Sleeping...");
    }
}

// Subclass providing implementation for abstract method
class Dog extends Animal {
    void makeSound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.makeSound(); // Calls overridden method
        d.sleep();     // Calls inherited method
    }
}
```

Output:

```
Dog barks
Sleeping...
```

2 Interfaces

An **interface** in Java is a blueprint for a class that contains **only abstract methods and constants** (before Java 8).

It is used for **achieving 100% abstraction** and **multiple inheritance**.

✓ Key Features:

- **Cannot contain instance variables** (only static & final variables).
- Methods are **public and abstract** by default.
- A class **implements** an interface using the **implements** keyword.

```
// Interface declaration
interface Vehicle {
    void start(); // Abstract method (no body)
}

// Class implementing the interface
class Car implements Vehicle {
    public void start() {
        System.out.println("Car is starting...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.start();
    }
}
```

Output:

```
Car is starting...
```

5.8) this & super Keywords in Java

1 this Keyword

The “**this**” keyword in Java refers to **the current instance of a class**.

✓ Uses of **this**:

- ✓ To differentiate between instance & local variables
- ✓ To call another constructor in the same class
- ✓ To return the current object

```
class Student {
    String name;

    // Using 'this' to differentiate between instance and local variable
    Student(String name) {
        this.name = name; // 'this.name' refers to the instance variable
    }

    void display() {
        System.out.println("Student Name: " + this.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Alice");
        s1.display();
    }
}
```

Output:

```
Student Name: Alice
```

2 **super** Keyword

The **super** keyword in Java is used to refer to the **parent class (superclass)**.

✓ Uses of **super**:

- ✓ Access parent class methods

- ✓ Access parent class constructor
- ✓ Access parent class variables

```
// Parent class
class Animal {
    String type = "Mammal";

    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

// Child class
class Dog extends Animal {
    String type = "Dog";

    void display() {
        System.out.println("Type in child class: " + this.type);
        System.out.println("Type in parent class: " + super.type); // Access parent class variable
    }

    void makeSound() {
        super.makeSound(); // Calls parent class method
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.display();
        d.makeSound();
    }
}
```

Output:

```
Type in child class: Dog
Type in parent class: Mammal
Animal makes a sound
Dog barks
```

5.9) Static Keyword in Java

The `static` keyword is used for **memory management**. It can be applied to **variables, methods, blocks, and nested classes**.

✓ Key Features:

- Static members belong to the class, not objects.
- Static methods can be called without creating an object.
- Static methods cannot access non-static (instance) variables directly.

```
class Counter {  
    // Static variable (shared among all objects)  
    static int count = 0;  
  
    Counter() {  
        count++;  
        System.out.println("Count: " + count);  
    }  
  
    // Static method  
    static void showCount() {  
        System.out.println("Total Objects Created: " + count);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Counter c1 = new Counter();  
        Counter c2 = new Counter();  
        Counter.showCount(); // Calling static method  
    }  
}
```

Output:

```
Count: 1  
Count: 2  
Total Objects Created: 2
```

Conclusion:

- ✓ **Classes & Objects** - Blueprint & instances
- ✓ **Constructors** - Initialize objects
- ✓ **Encapsulation** - Data hiding
- ✓ **Inheritance** - Parent-child relationship
- ✓ **Polymorphism** - Overloading & Overriding
- ✓ **Abstract Classes & Interfaces** - Blueprint without full implementation
- ✓ **this & super** - Referring to objects and parent classes

🚀 **Next, we'll explore Arrays and Strings in Java!**

Pluto Academy

Chapter 6. Arrays and Strings in Java

Arrays and Strings are essential concepts in Java that help store and manipulate data efficiently.

- **Arrays:** Used to store multiple values of the same type.
- **Strings:** A sequence of characters, commonly used for text processing.

6.1) Introduction to Arrays

What is an Array?

An **array** is a **fixed-size container** that holds multiple values of the **same data type**.

Real-life Analogy:

Think of an **egg tray**—it has multiple slots (elements) of the same size and type (eggs). Each slot is identified by an **index number** starting from 0.

Why use Arrays ?

- ✓ Efficient way to store and access multiple elements
- ✓ Reduces redundant variable declarations
- ✓ Helps in easy iteration and manipulation

Syntax:

```
// Syntax for declaring an array
datatype[] arrayName = new datatype[size];
```

Example: Declaring and Initializing an Array

```
public class ArrayExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50}; // Initializing an array
        System.out.println("First Element: " + numbers[0]);
    }
}
```

Output:

```
First Element: 10
```

6.2) 1D and 2D Arrays

🔗 One-Dimensional (1D) Arrays

A **1D array** is a list of elements stored in a single row.

Example: Creating and Accessing a 1D Array

```
public class OneDArray {
    public static void main(String[] args) {
        int[] marks = new int[3]; // Array of size 3
        marks[0] = 85;
        marks[1] = 90;
        marks[2] = 78;

        // Printing array elements
        for (int i = 0; i < marks.length; i++) {
            System.out.println("Marks[" + i + "]: " + marks[i]);
        }
    }
}
```

Output:

```
Marks[0]: 85  
Marks[1]: 90  
Marks[2]: 78
```

Two-Dimensional (2D) Arrays

A **2D array** is like a table or matrix, with rows and columns.

Real-life Analogy:

Think of a **chessboard**—each cell has a row and column index.

Example: Declaring a 2D Array

```
public class TwoDArray {  
    public static void main(String[] args) {  
        int[][] matrix = { {1, 2, 3}, {4, 5, 6} };  
  
        // Accessing elements  
        System.out.println("Element at (0,1): " + matrix[0][1]);  
        System.out.println("Element at (1,2): " + matrix[1][2]);  
    }  
}
```

Output:

```
Element at (0,1): 2  
Element at (1,2): 6
```

6.3) Array Operations (Sorting, Searching, Copying)

Sorting an Array

We can use **Arrays.sort()** to sort elements in ascending order.

```
import java.util.Arrays;

public class SortArray {
    public static void main(String[] args) {
        int[] numbers = {5, 1, 8, 3, 2};
        Arrays.sort(numbers); // Sorting array
        System.out.println(Arrays.toString(numbers)); // Output sorted array
    }
}
```

Output:

```
[1, 2, 3, 5, 8]
```

Searching an Element in an Array

We can use **Arrays.binarySearch()** to find an element.

```
import java.util.Arrays;

public class SearchArray {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int index = Arrays.binarySearch(arr, 30); // Search for 30
        System.out.println("Element found at index: " + index);
    }
}
```

Output:

```
Element found at index: 2
```

Copying an Array

```

public class CopyArray {
    public static void main(String[] args) {
        int[] original = {1, 2, 3};
        int[] copy = Arrays.copyOf(original, original.length);
        System.out.println("Copied Array: " + Arrays.toString(copy));
    }
}

```

Output:

```
Copied Array: [1, 2, 3]
```

Note: These are just a few methods shown but Java provides lots of methods so do search on the internet.

6.4) String Class & Methods

A **String** is a sequence of characters enclosed in double quotes (" "). Unlike arrays, Strings in Java are **immutable** (cannot be changed after creation).

Example: Creating a String

```

public class StringExample {
    public static void main(String[] args) {
        String name = "Java Programming";
        System.out.println("Length: " + name.length());
        System.out.println("Uppercase: " + name.toUpperCase());
    }
}

```

Output:

```

Length: 16
Uppercase: JAVA PROGRAMMING

```

6.5) Array Methods in Java

Method	Description
<code>Arrays.toString(array)</code>	Converts an array into a String representation.
<code>Arrays.sort(array)</code>	Sorts an array in ascending order.
<code>Arrays.binarySearch(array, key)</code>	Searches for a key using binary search (only works on sorted arrays).
<code>Arrays.equals(arr1, arr2)</code>	Compares two arrays for equality.
<code>Arrays.fill(array, value)</code>	Fills the entire array with a specified value.
<code>Arrays.copyOf(arr, newLength)</code>	Copies the specified array to a new array with the given length.
<code>Arrays.copyOfRange(arr, start, end)</code>	Copies a range of elements from an array.
<code>Arrays.asList(arr)</code>	Converts an array into a List.
<code>Arrays.stream(arr)</code>	Converts an array into a Stream (for functional programming).
<code>System.arraycopy(src, srcPos, dest, destPos, length)</code>	Copies elements from one array to another.

6.6) String Methods in Java

Method	Description
<code>str.length()</code>	Returns the length of the string.
<code>str.charAt(index)</code>	Returns the character at the specified index.
<code>str.concat(str2)</code>	Concatenates two strings.
<code>str.toLowerCase()</code>	Converts all characters to lowercase.
<code>str.toUpperCase()</code>	Converts all characters to uppercase.
<code>str.trim()</code>	Removes leading and trailing spaces.
<code>str.substring(start, end)</code>	Extracts a substring from the given indices.
<code>str.indexOf(ch)</code>	Returns the index of the first occurrence of the character.
<code>str.lastIndexOf(ch)</code>	Returns the index of the last occurrence of the character.
<code>str.equals(str2)</code>	Compares two strings for exact equality.
<code>str.equalsIgnoreCase(str2)</code>	Compares two strings, ignoring case differences.
<code>str.contains(substring)</code>	Checks if the string contains a specific substring.
<code>str.replace(oldChar, newChar)</code>	Replaces all occurrences of a character.
<code>str.replaceAll(regex, replacement)</code>	Replaces all occurrences based on regex.
<code>str.split(regex)</code>	Splits the string into an array based on a regex pattern.
<code>str.startsWith(prefix)</code>	Checks if the string starts with a specific prefix.
<code>str.endsWith(suffix)</code>	Checks if the string ends with a specific suffix.

Bonus: Mixed Array and String Question!

◆ Easy Level Questions

1 Array Questions (Easy)

1. Find the Largest Element in an Array
2. Find the Second Largest Element in an Array

3. Reverse an Array
 4. Find Missing Number in an Array (1 to N)
 5. Find the Duplicate Element in an Array
 6. Find the Intersection of Two Arrays
 7. Check if an Array is Sorted
 8. Move Zeroes to End of Array
 9. Merge Two Sorted Arrays
 10. Left Rotate an Array by One Position
-

2 String Questions (Easy)

1. Check if a String is a Palindrome
2. Reverse a String
3. Count the Number of Vowels and Consonants in a String
4. Find the First Non-Repeating Character in a String
5. Check if Two Strings are Anagrams
6. Convert a String to Lowercase or Uppercase
7. Find the Frequency of a Character in a String
8. Remove All Spaces from a String
9. Check if a String is a Valid Pangram (Contains all alphabets A-Z)

10. Replace All Spaces in a String with '%20' (Like URL Encoding)
-

◆ Medium Level Questions

1 Array Questions (Medium)

1. Find the Majority Element in an Array (Appears more than $N/2$ times)
 2. Find the Subarray with the Maximum Sum (Kadane's Algorithm)
 3. Sort an Array of 0s, 1s, and 2s (Dutch National Flag Problem)
 4. Find the Longest Consecutive Sequence in an Array
 5. Find the Two Numbers in an Array that Add Up to a Given Target (Two Sum)
 6. Rearrange an Array in Alternating Positive and Negative Numbers
 7. Find the Kth Largest Element in an Array
 8. Rotate an Array by K Positions (Cyclic Rotation)
 9. Find the Smallest Missing Positive Integer
 10. Find the Union and Intersection of Two Arrays
-

2 String Questions (Medium)

1. Find the Longest Substring Without Repeating Characters
2. Find the Longest Palindromic Substring

3. Check if One String is a Rotation of Another
4. Find the First Non-Repeating Word in a Sentence
5. Group Anagrams from an Array of Strings
6. Find the Minimum Window Substring Containing All Characters of Another String
7. Find the Longest Common Prefix in an Array of Strings
8. Check if Two Strings are One Edit Distance Apart
9. Reverse Words in a Sentence Without Using Extra Space
10. Implement String Compression (Like "aabbcc" → "a2b2c2")

Note: For more details about the specific question go for coding platforms like Leetcode, CodeForces, HackerRank, CodeChef etc.

Conclusion:

- ✓ **Arrays** - 1D and 2D arrays for structured data storage
- ✓ **Sorting, Searching, Copying** - Essential operations for handling arrays
- ✓ **Strings** - Immutable character sequences
- ✓ **StringBuffer & StringBuilder** - For efficient string manipulations
- ✓ **Common String Problems** - Finding, replacing, and checking substrings

🚀 **Next, we'll explore Exception Handling in Java!**

Chapter 7. Exception Handling in Java

Introduction to Exception Handling

🔗 What is Exception Handling?

In Java, exception handling is a mechanism that helps us detect and handle runtime errors gracefully instead of abruptly terminating the program.

💡 Real-Life Analogy:

Imagine you're watching a YouTube video, and suddenly your internet disconnects. Instead of crashing, YouTube shows a "No Internet Connection" message. This is similar to exception handling in Java—when an error occurs, Java provides an alternative way to handle it.

Why Exception Handling ?

- ✓ Prevents program crashes
- ✓ Helps in debugging and locating errors easily
- ✓ Ensures smooth execution of programs
- ✓ Allows developers to create custom error messages

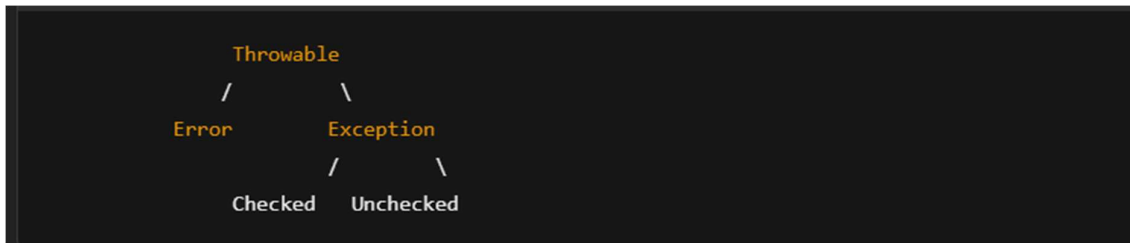
7.1) Types of error in Java:

Java errors are categorized into three main types:

Type of Error	Description	Example
Syntax Errors	Errors due to incorrect Java syntax, found at compile time.	Missing semicolon ;
Logical Errors	Program runs but produces incorrect results due to bad logic.	<pre>if (x > y) { y = x; }</pre> (wrong logic)
Runtime Errors (Exceptions)	Errors that occur during program execution, causing it to crash.	Dividing by zero <pre>int x = 5 / 0;</pre>

Exception Hierarchy in Java

In Java, all exceptions are subclasses of the `Throwable` class.



1 Checked Exceptions (Compile-time Exceptions)

These exceptions must be handled using `try-catch` or `throws`, or the program won't compile.

◆ Examples:

- `IOException` (File not found)
- `SQLException` (Database issues)
- `InterruptedException` (Thread interruption)

2 Unchecked Exceptions (Runtime Exceptions)

These exceptions occur during program execution and are caused by logical mistakes.

◆ Examples:

- `ArithmeticException` (Dividing by zero)
- `NullPointerException` (Accessing null objects)

- **ArrayIndexOutOfBoundsException** (Accessing invalid array index)

7.2) Handling Exceptions using **try**, **catch**, and **finally** Blocks

✂ The **try** Block

The try block contains the code that may cause an exception.

✂ The **catch** Block

The catch block handles the exception if it occurs.

✂ The **finally** Block

The finally block always executes, regardless of whether an exception occurs or not.

Example: Handling Division by Zero Exception

```
public class TryCatchExample {  
    public static void main(String[] args) {  
        try {  
            // Risky code that may cause an exception  
            int result = 10 / 0;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            // Handling the exception  
            System.out.println("Error: Cannot divide by zero!");  
        } finally {  
            // This block always executes  
            System.out.println("Execution completed.");  
        }  
    }  
}
```

Output:

```
Error: Cannot divide by zero!  
Execution completed.
```

📌 Explanation:

- The `try` block attempts to divide by zero, which causes an `ArithmeticException`.
- The `catch` block handles this exception and prints an error message.
- The `finally` block executes no matter what happens.

7.3) Multiple Catch Blocks

A single `try` block can have **multiple catch blocks** to handle different exceptions separately.

Example: Handling Multiple Exceptions

```
public class MultipleCatchExample {  
    public static void main(String[] args) {  
        try {  
            int[] numbers = {10, 20, 30};  
            System.out.println(numbers[5]); // Out of bounds access  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Error: Array index is out of bounds!");  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Arithmetic Exception occurred!");  
        } catch (Exception e) {  
            System.out.println("Error: Some other exception occurred!");  
        }  
    }  
}
```

Output:

```
Error: Array index is out of bounds!
```

7.4) throw and throws Keyword

(1) throw Keyword

The **throw** keyword is used to manually throw an exception.

Example: Throwing an Exception Manually

```
public class ThrowExample {  
    public static void validateAge(int age) {  
        if (age < 18) {  
            throw new ArithmeticException("Access denied: You must be 18 or older.");  
        } else {  
            System.out.println("Access granted!");  
        }  
    }  
  
    public static void main(String[] args) {  
        validateAge(16); // This will throw an exception  
    }  
}
```

Output: Exception in thread "main" java.lang.ArithmeticException: Access denied: You must be 18 or older.

Explanation:

- If `age < 18`, we manually throw an `ArithmeticException`.
- The program stops execution when “**throw**” is encountered.

(2) throws Keyword

The **throws** keyword is used in method declarations to specify that the method may throw an exception.

Example: Using **throws** in a Method

```
import java.io.*;

public class ThrowsExample {
    static void readFile() throws IOException {
        FileReader file = new FileReader("nonexistent.txt"); // File not found
        BufferedReader fileInput = new BufferedReader(file);
    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("Error: File not found!");
        }
    }
}
```

Output:

```
Error: File not found!
```

Explanation:

- The **readFile()** method declares that it may throw an **IOException**.
- The **main()** method handles this exception using **try-catch**.

7.5) Custom Exceptions in Java

Java allows us to create **custom exceptions** by extending the **Exception** class.

Example: Creating and Throwing a Custom Exception

```
// Custom Exception class
class InvalidMarksException extends Exception {
    public InvalidMarksException(String message) {
        super(message);
    }
}

public class CustomExceptionExample {
    static void checkMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Invalid marks: Must be between 0 and 100.");
        } else {
            System.out.println("Valid Marks: " + marks);
        }
    }

    public static void main(String[] args) {
        try {
            checkMarks(150); // Invalid marks
        } catch (InvalidMarksException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

Output:

```
Exception: Invalid marks: Must be between 0 and 100.
```

Explanation:

- We created a **InvalidMarksException** class that extends **Exception**.
- The **checkMarks()** method throws this exception when invalid marks are given.

Note: To create a custom exception the user need to extends a class "Exception" then only user can able to create its own custom error handler.

Conclusion:

- ✅ Errors in Java - Syntax, Logical, and Runtime Errors
- ✅ Exception Handling - `try`, `catch`, `finally`
- ✅ Multiple Catch Blocks - Handling multiple exceptions
- ✅ `throw` and `throws` - Manually throwing and declaring exceptions
- ✅ Custom Exceptions - Creating user-defined exceptions

🚀 **Next, we will explore Java Collection Framework!**

Pluto Academy

Chapter 8. Java Collections Framework (JCF)

8.1) Introduction to Java Collections Framework (JCF)

What is the Java Collections Framework?

The Java Collections Framework (JCF) is a set of predefined classes and interfaces in Java that help in storing, processing, and manipulating groups of objects efficiently.

Why Use Collections Instead of Arrays?

Feature	Arrays	Collections
Size	Fixed	Dynamic (Can grow/shrink)
Data Type	Only one type (homogeneous)	Can store different types (heterogeneous)
Performance	Fast for primitive types	More flexible but slightly slower
Built-in Methods	Limited	Rich built-in functionalities like sorting, searching, etc.

Key Interfaces in Java Collections Framework

Java provides three main interfaces in JCF:

Interface	Description	Common Implementations
List	Ordered collection, allows duplicates	ArrayList , LinkedList
Set	Unordered, unique elements	HashSet , TreeSet
Map	Key-value pairs, unique keys	HashMap , TreeMap

8.2) List Interface (Ordered Collection with Duplicates)

The **List** interface allows elements to be stored in a sequence and permits duplicates.

When to Use ?

- When order matters (e.g., storing student roll numbers).

- When duplicate values are required (e.g., storing shopping cart items).

Common Implementations of List:

Implementation	Features	When to Use?
ArrayList	Fast random access, slow insert/delete	When frequent reads are needed
LinkedList	Fast insert/delete, slow access	When frequent insertions/removals are needed

◆ Example: Using ArrayList

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();

        // Adding elements
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        // Accessing elements
        System.out.println("First element: " + names.get(0));

        // Iterating using for-each
        for (String name : names) {
            System.out.println(name);
        }
    }
}
```

Output:

```
First element: Alice
Alice
Bob
Charlie
```

📌 Explanation:

- We use `ArrayList` because we need fast access to elements using indices.

- Elements maintain their insertion order.

◆ Example: Using LinkedList

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Dog");
        list.add("Cat");
        list.add("Elephant");

        System.out.println(list); // Output: [Dog, Cat, Elephant]
    }
}
```

Comparison:

Feature	ArrayList	LinkedList
Structure	Uses dynamic array	Uses doubly linked list
Access Speed	Fast ($O(1)$)	Slow ($O(n)$)
Insertion/Deletion	Slow ($O(n)$)	Fast ($O(1)$)
Memory Usage	Less (stores only values)	More (stores values + pointers)

◇ Set Interface

A Set is a collection of unique elements (no duplicates). It does not maintain insertion order.

✓ Implementations:

- **HashSet** (No ordering)
- **TreeSet** (Sorted order)

Comparison:

Feature	HashSet	TreeSet
Ordering	No ordering	Sorted in natural order
Performance	Faster ($O(1)$)	Slower ($O(\log n)$)
Null Values	Allows one null	No null values

Example: HashSet

```
import java.util.HashSet;

public class SetExample {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
        set.add(5);
        set.add(1);
        set.add(10);
        set.add(5); // Duplicate, ignored

        System.out.println(set); // Output: [1, 5, 10] (order may vary)
    }
}
```

Note: TreeSet works in the same fashion.

📌 HashMap vs TreeMap

- Both store key-value pairs, but they differ in ordering and performance.

Feature	HashMap	TreeMap
Ordering	No ordering	Sorted by keys
Performance	Fast ($O(1)$)	Slower ($O(\log n)$)
Null Keys	Allows one null	No null keys

♦ Example: Using HashMap

```
java Copy Edit

import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(3, "Apple");
        map.put(1, "Banana");
        map.put(2, "Cherry");

        System.out.println(map); // Output: {1=Banana, 2=Cherry, 3=Apple} (order may vary)
    }
}
```

♦ Example: Using TreeMap

```
java Copy Edit

import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {
        TreeMap<Integer, String> map = new TreeMap<>();
        map.put(3, "Apple");
        map.put(1, "Banana");
        map.put(2, "Cherry");

        System.out.println(map); // Output: {1=Banana, 2=Cherry, 3=Apple} (sorted order)
    }
}
```

Conclusion:

In this chapter, you have learned:

✓ **Introduction to Collections** – The Java Collections Framework (JCF) provides a unified architecture for storing and manipulating data efficiently.

✅ **List, Set, Map Interfaces** – You explored the three main collection interfaces:

- **List** (Ordered, allows duplicates)
- **Set** (Unordered, no duplicates)
- **Map** (Key-Value pairs)

✅ **ArrayList vs LinkedList** – You understood the difference:

- **ArrayList** – Fast for indexing but slow for insertions/deletions in the middle.
- **LinkedList** – Efficient for insertions/deletions but slower for random access.

✅ **HashSet vs TreeSet** – You compared:

- **HashSet** – Fast, unordered collection with unique elements.
- **TreeSet** – Sorted collection but slightly slower due to sorting overhead.

✅ **HashMap vs TreeMap** – You explored:

- **HashMap** – Unordered key-value storage with fast operations.
- **TreeMap** – Stores key-value pairs in sorted order (by keys).

🚀 **Next, we will explore Multithreading in Java!**

Chapter 9. Multithreading & Concurrency

9.1) Introduction to Threads

What is a Thread?

A **thread** is the smallest unit of execution in a program. In Java, multithreading allows multiple threads to run concurrently within a program, sharing resources and performing different tasks. Multithreading is used to **perform multiple operations simultaneously**, increasing the performance of applications, especially on multi-core processors.

Why Use Multithreading?

- **Concurrency:** Multithreading enables efficient multitasking, allowing multiple tasks to be processed at the same time.
- **Better Resource Utilization:** Threads share the same memory and resources, making it more efficient compared to creating new processes for each task.
- **Responsiveness:** Multithreading can make applications more responsive, as one thread can handle user input while another processes data in the background.

9.2) Creating Threads in Java

There are **two primary ways** to create threads in Java:

1. Extending the **Thread** Class

You can create a thread by **extending the **Thread** class** and overriding its **run()** method, which contains the code that will be executed by the thread.

Example: Creating a Thread by Extending **Thread Class**

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }

    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // Starting the thread
    }
}

```

Output:

```
Thread is running...
```

📌 Explanation:

- The `run()` method contains the logic that the thread will execute.
- The `start()` method is used to begin the execution of the thread. It internally calls the `run()` method.

2. Implementing the `Runnable` Interface

Another way to create a thread is by **implementing the `Runnable` interface**. This approach is preferred when your class is already extending another class (since Java supports single inheritance only).

Example: Creating a Thread by Implementing `Runnable` Interface

```

class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running...");
    }

    public static void main(String[] args) {
        MyRunnable myRunnable = new MyRunnable();
        Thread thread = new Thread(myRunnable); // Passing Runnable to Thread
        thread.start(); // Starting the thread
    }
}

```

Output:

```
Runnable thread is running...
```

📌 Explanation:

- The `run()` method contains the logic to be executed by the thread.
- The `Thread` constructor accepts a `Runnable` object, and the `start()` method initiates the thread.

When to Use Which ?

- **Extend `Thread`:** If the class does not need to extend any other class, this is the simpler approach.
- **Implement `Runnable`:** Use this if your class already extends another class or if you want to share the same `Runnable` object between multiple threads.

9.3) Thread Lifecycle

A thread in Java undergoes the following states during its lifecycle:

Thread States:

1. **New:** A thread is in the "new" state when it is created but not yet started.
2. **Runnable:** A thread enters this state after invoking the `start()` method. It is eligible for execution by the thread scheduler but may not be running immediately.
3. **Blocked:** A thread enters the "blocked" state when it is waiting for a resource that is currently unavailable (e.g., waiting for I/O operations or waiting to acquire a lock).

4. **Waiting:** A thread enters this state when it is waiting indefinitely for another thread to perform a particular action.
5. **Timed Waiting:** A thread is in this state when it is waiting for a specific period of time.
6. **Terminated:** A thread is in the "terminated" state when it completes its execution or is terminated due to an exception.

9.4) Synchronization in Java

🔑 What is Synchronization ?

Synchronization is a mechanism in Java that ensures that multiple threads do not interfere with each other while accessing shared resources (e.g., variables, methods). This is crucial to avoid **race conditions** where two or more threads modify shared data simultaneously, leading to inconsistent results.

Synchronized Methods

- A method can be synchronized by using the **synchronized** keyword, ensuring that only one thread can execute the method at any given time.

Example: Synchronized Method

```
class Counter {  
    private int count = 0;  
  
    // Synchronized method to ensure thread-safe increment  
    public synchronized void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```

```

public class SynchronizationExample {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();

        // Creating two threads
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });

        // Start both threads
        t1.start();
        t2.start();

        // Wait for both threads to finish
        t1.join();
        t2.join();

        // Output the final count
        System.out.println("Final count: " + counter.getCount());
    }
}

```

Note: The above two image is a single code

Output:

```
Final count: 2000
```

Conclusion:

- ✓ You have learned about **Threads in Java**, understanding their importance in enabling concurrent execution.
- ✓ You now know how to **create threads** using both the **Thread** class and the **Runnable** interface, giving you flexibility in multithreading.

✅ You've explored the **Thread Lifecycle**, which helps you understand the various states a thread can be in during its execution.

✅ You've learned about **Synchronization**, ensuring safe access to shared resources between threads to avoid potential data inconsistency.

🚀 **Next, we will move to our last chapter of this course!**

Pluto Academy

Chapter 10. Future of Java & Career Path

Java has been one of the most popular and widely used programming languages for over two decades, and its future remains promising. With constant updates, a vibrant developer community, and a broad range of applications, Java continues to evolve and adapt to meet the demands of modern software development. In this chapter, we'll explore the future trends of Java, its use across industries, the career roadmap for Java developers, certifications, and resources to master Java.

10.1) Java Trends & Future Scope

Java has long been known for its stability, portability, and scalability, making it the go-to language for enterprise applications. Here are some trends and future scope areas where Java is expected to continue thriving:

1. Java in Cloud Computing

- Cloud services like AWS, Google Cloud, and Microsoft Azure are heavily relying on Java for building scalable cloud-based applications. With its platform independence, Java continues to be a strong contender in the cloud-native development ecosystem. Java is well-suited for microservices architecture, which is integral to cloud computing.

2. Integration with Big Data and Machine Learning

- Java has a strong ecosystem for data-driven applications. Frameworks like Hadoop and Apache Spark support Java for processing large datasets. Additionally, Java libraries such as Weka, Deeplearning4j, and Apache Mahout are gaining popularity in the Machine Learning domain. As data science and AI continue to grow, Java's ability to

handle big data and complex machine learning models ensures its relevance.

3. Java and IoT (Internet of Things)

- The rise of IoT brings new opportunities for Java. Java's portability and security make it a great fit for IoT applications, ranging from small devices to large systems. With the increasing need for connected devices, Java's flexibility and cross-platform capabilities make it an ideal choice for building IoT solutions.

4. Modern Java for Mobile and Desktop Development

- While Kotlin has emerged as the preferred language for Android development, Java remains relevant in mobile application development, especially for legacy projects. JavaFX is used for developing desktop applications with rich user interfaces, and Java's integration with cross-platform frameworks continues to grow.

5. Java for Microservices

- Microservices architecture, which focuses on building small, modular, and independently deployable services, has gained significant traction. Java frameworks like Spring Boot and Quarkus allow developers to build microservices with ease. The Spring Cloud ecosystem and Kubernetes integration ensure seamless deployment and management of Java-based microservices in the cloud.

6. Performance Improvements

- Java's performance has improved significantly with the latest releases. The Java Virtual Machine (JVM) continues to be optimized for better memory management, garbage collection, and performance. The new JEPs (JVM Enhancement Proposals) and features like JEP 355 (Text Blocks), JEP 358 (Helpful NullPointerException), and JEP 359 (Records) show that Java continues to evolve and improve with each new release.

7. Adoption of GraalVM

- GraalVM is a high-performance runtime that provides support for multiple languages (Java, JavaScript, Python, etc.). Java developers can use GraalVM to compile Java code into native executables, resulting in faster startup times and reduced memory usage. This is particularly beneficial for cloud-native applications and microservices.

8. Java's Role in Enterprise Solutions

- Java is well-established as the language of choice for large-scale enterprise applications. With technologies like Spring Framework, Hibernate, and Java EE (Jakarta EE), Java continues to dominate the enterprise space. As companies continue to modernize their systems, Java remains an essential part of the enterprise ecosystem.
-

10.2) Industries Using Java

Java is ubiquitous across many industries due to its reliability, scalability, and flexibility. Some of the key industries that rely on Java include:

1. Finance and Banking

- Java is widely used in banking software, payment processing systems, and trading platforms due to its robustness and ability to handle high volumes of transactions. Financial institutions rely on Java for real-time trading, risk management systems, and banking applications.

2. E-commerce

- Major e-commerce platforms, including Amazon, eBay, and Alibaba, rely on Java for building their backend systems. Java provides the necessary scalability, reliability, and security required to handle millions of transactions per day.

3. Telecommunications

- Java is the foundation for telecom software systems, including call routing, billing systems, and customer management systems. Telecom giants use Java for both network management and data analytics.

4. Healthcare

- Healthcare companies use Java to build electronic health record (EHR) systems, patient management applications, and medical device software. Java's security features are crucial in healthcare, where patient data must be protected and easily shared between systems.

5. Retail & Manufacturing

- ERP (Enterprise Resource Planning) systems and supply chain management software are often built using Java. Java's integration capabilities allow retail and manufacturing companies to streamline operations, manage inventories, and analyze data in real-time.

6. Government

- Java is widely used in the public sector for building secure, scalable, and highly available applications. Government agencies around the world trust Java for systems related to taxation, citizen services, and defense applications.

7. Gaming

- While C++ and Unity are commonly used in game development, Java is used for building mobile games (especially Android games). Minecraft, one of the most popular games of all time, is built using Java.
-

10.3) Java Developer Career Roadmap

Becoming a Java Developer requires a well-planned career path, a strong foundation in core Java concepts, and hands-on experience with modern frameworks and tools. Here's a roadmap to help you get started:

1. Master the Basics

- The first step to becoming a Java developer is to learn the fundamentals of the language, including syntax, data types, loops, conditionals, methods, classes, and object-oriented programming principles. Once you've grasped these concepts, you can move on to more advanced topics.

2. Work with Java Frameworks

- Java frameworks such as Spring Boot, Hibernate, and JavaFX are essential for developing modern Java applications. Familiarizing yourself with these frameworks will set you apart from other developers and help you build robust, scalable applications.

3. Learn Databases and SQL

- Understanding how to interact with databases is crucial for a Java developer. Learn SQL and become proficient in working with relational databases like MySQL and PostgreSQL, as well as NoSQL databases like MongoDB. Understand JDBC (Java Database Connectivity) to connect your Java applications to databases.

4. Get Hands-On Experience with Web Development

- Learn Servlets and JSP for building web applications, and get familiar with modern web frameworks like Spring MVC and Spring Boot. Knowledge of RESTful APIs and JSON is essential for integrating backend services with frontend applications.

5. Understand Build Tools and Version Control

- Learn how to use build tools like Maven and Gradle for automating your project's build process. Additionally, get comfortable with Git for version control, as it is an essential tool for collaborative development.

6. Focus on Testing and Quality

- Writing testable code is critical for long-term software maintenance. Learn how to use JUnit and Mockito for unit testing and integration testing. Familiarize yourself with TDD (Test-Driven Development) and CI/CD (Continuous Integration/Continuous Deployment) pipelines to ensure high-quality code.

7. Learn Cloud and DevOps

- Java developers should be comfortable with cloud computing platforms like AWS, Google Cloud, or Microsoft Azure. Understanding Docker, Kubernetes, and CI/CD pipelines will help you deploy and manage Java applications in the cloud.

8. Stay Updated with New Features

- Java is constantly evolving, with new features introduced in every version. Stay updated with the latest Java versions (e.g., Java 17, Java 18) and the features they bring, such as records, sealed classes, text blocks, and more.

10.4) Certifications for Java Developers

Java certifications are an excellent way to validate your skills and showcase your expertise to potential employers. Here are some popular Java certifications:

1. Oracle Certified Associate (OCA)

- The OCA Java SE Programmer certification is ideal for beginners and focuses on core Java programming concepts.

2. Oracle Certified Professional (OCP)

- The OCP Java SE Programmer certification is for intermediate to advanced Java developers, covering deeper aspects of Java, such as multi-threading, collections, and exception handling.

3. Spring Professional Certification

- For developers working with the Spring Framework, the Spring Professional Certification validates your ability to build Java applications using the Spring ecosystem.

4. AWS Certified Developer – Associate

- This certification is for developers working with Java in cloud environments. It validates your skills in developing and deploying applications on AWS using Java.

5. Java EE 7/8 Certification

- Java EE (now Jakarta EE) certifications focus on enterprise Java applications. These certifications are ideal for developers working with web and enterprise applications.

10.5) Resources & Best Practices to Master Java

Here are some key resources and best practices to master Java:

1. Books

- Effective Java by Joshua Bloch – A must-read for Java developers looking to improve their coding practices.

- Java: The Complete Reference by Herbert Schildt – A comprehensive guide to Java, covering everything from the basics to advanced topics.

2. Online Courses

- Udemy, Coursera, and Pluralsight offer a variety of Java courses for beginners and advanced learners.
- Java Programming and Software Engineering Fundamentals by Duke University (Coursera) is a well-structured learning path.

3. Practice Platforms

- Platforms like LeetCode, HackerRank, and CodeSignal offer coding challenges to improve your problem-solving skills.

4. Community

- Join the Java community on forums like Stack Overflow, Reddit's r/Java, and GitHub. Participating in open-source projects is a great way to learn from experienced developers and contribute to real-world applications.

5. Best Practices

- Always write clean, readable, and maintainable code.
 - Follow design patterns, such as Singleton, Factory, and Observer, to solve common problems.
 - Use proper exception handling to deal with errors gracefully.
 - Write unit tests and follow Test-Driven Development (TDD) principles.
-

Conclusion:

- ✓ Java Trends – Cloud computing, big data, IoT, microservices, and more.
- ✓ Industries Using Java – From finance to healthcare, Java is essential.
- ✓ Career Roadmap – Master Java fundamentals, frameworks, and cloud tools.
- ✓ Certifications – Validate your skills with Oracle, Spring, and AWS certifications.
- ✓ Resources – Learn through books, courses, coding platforms, and communities.

Pluto Academy

Bonus Section: MCQs & Interview Questions

1. Java Basics (20 MCQs)

1. What is the default value of a local variable in Java?

- a) null
- b) 0
- c) undefined
- d) Local variables do not have a default value

Answer: d) Local variables do not have a default value

2. Which of the following is the correct way to declare a variable in Java?

- a) `int 1a = 5;`
- b) `int a1 = 5;`
- c) `int = 5 a1;`
- d) `1a = 5;`

Answer: b) `int a1 = 5;`

3. Which of these is not a valid keyword in Java?

- a) static
- b) void
- c) int
- d) number

Answer: d) number

4. Which operator is used to compare two values in Java?

- a) ==
- b) =
- c) ===
- d) !=

Answer: a) ==

5. What is the output of the following code?

```
int x = 5;  
int y = 10;  
System.out.println(x + y);
```

- a) 510
- b) 15
- c) 5

- d) 10

Answer: b) 15

6. Which of the following is a primitive data type in Java?

- a) String
- b) Integer
- c) float
- d) Object

Answer: c) float

7. Which of these statements is true about Java?

- a) Java is a platform-dependent language
- b) Java is platform-independent
- c) Java is a low-level programming language
- d) Java can only run on Windows

Answer: b) Java is platform-independent

8. What will be the output of the following code?

```
int a = 8;  
int b = 4;  
System.out.println(a / b);
```

- a) 4
- b) 2
- c) 3.0
- d) 3

Answer: b) 2

9. Which of the following methods can be used to find the length of a string in Java?

- a) length()
- b) size()
- c) getLength()
- d) getSize()

Answer: a) length()

10. Which of the following is not an access modifier in Java?

- a) public
- b) private
- c) protected
- d) secured

Answer: d) secured

11. What is the default value of a boolean variable in Java?

- a) 0
- b) false
- c) true
- d) null

Answer: b) false

12. Which of the following statements is correct about the **String** class in Java?

- a) String is a primitive data type
- b) String is immutable in Java
- c) String can be changed during runtime
- d) String objects are mutable

Answer: b) String is immutable in Java

13. What does the **break** statement do in a loop?

- a) Exits the loop and continues to the next iteration
- b) Exits the loop entirely
- c) Skips the rest of the loop and continues with the next iteration
- d) Stops the program execution

Answer: b) Exits the loop entirely

14. Which of these is used to handle exceptions in Java?

- a) try-catch
- b) if-else
- c) switch-case
- d) continue

Answer: a) try-catch

15. Which of the following is the correct syntax to create an array in Java?

- a) `int arr[] = new int[5];`
- b) `int arr = new int(5);`
- c) `int[] arr = new int(5);`
- d) `array arr[5] = new int();`

Answer: a) `int arr[] = new int[5];`

16. Which of the following methods is used to start a thread in Java?

- a) run()
- b) start()
- c) execute()
- d) init()

Answer: b) start()

17. Which of these classes is used for I/O in Java?

- a) FileReader
- b) StringReader
- c) BufferedReader
- d) FileWriter

Answer: a) FileReader

18. What is the size of an `int` in Java?

- a) 16 bits
- b) 32 bits
- c) 64 bits
- d) 8 bits

Answer: b) 32 bits

19. Which of the following is the default value of an object reference variable in Java?

- a) null
- b) 0
- c) Undefined

- d) false

Answer: a) null

20. Which of the following is not a valid method to declare an array in Java?

- a) `int arr[] = {1, 2, 3};`
- b) `int[] arr = new int[]{1, 2, 3};`
- c) `int arr[] = new int[3];`
- d) `int arr = {1, 2, 3};`

Answer: d) `int arr = {1, 2, 3};`

2. OOP Concepts (20 MCQs)

1. Which of the following is not an OOP concept in Java?

- a) Abstraction
- b) Encapsulation
- c) Compilation
- d) Polymorphism

Answer: c) Compilation

2. What is the concept of Inheritance in Java?

- a) Creating new classes from existing ones
- b) Hiding the implementation details of a class
- c) Modifying methods of the parent class
- d) Giving access to properties of other classes

Answer: a) Creating new classes from existing ones

3. What does the keyword **super** refer to in Java?

- a) It is used to access methods of the current class
- b) It is used to access parent class methods and variables
- c) It is used to create a new class
- d) It refers to the object in the current class

Answer: b) It is used to access parent class methods and variables

4. What is the purpose of an abstract class in Java?

- a) To allow an object to be created without any implementation

- b) To define methods that can be inherited by subclasses
- c) To provide a blueprint for child classes
- d) All of the above

Answer: d) All of the above

5. What is the primary benefit of encapsulation in Java?

- a) It improves performance by using fewer classes
- b) It hides the complexity and details of the implementation from the user
- c) It allows the programmer to work only with a single class
- d) It allows the inheritance of classes

Answer: b) It hides the complexity and details of the implementation from the user

6. Which of the following defines Polymorphism in Java?

- a) The ability of an object to take many forms
- b) The ability to reuse a class
- c) The ability to override a method in the parent class

- d) The ability to handle multiple exceptions

Answer: a) The ability of an object to take many forms

7. Which of these is true about an interface in Java?

- a) Interfaces can have method implementations
- b) Interface methods are abstract by default
- c) An interface can have instance variables
- d) A class can inherit multiple interfaces

Answer: b) Interface methods are abstract by default

8. Which of the following is not a feature of abstract classes in Java?

- a) Abstract classes can have both abstract and non-abstract methods
- b) An abstract class can have a constructor
- c) Abstract classes cannot be instantiated
- d) Abstract classes cannot have fields or variables

Answer: d) Abstract classes cannot have fields or variables

9. What does the keyword `this` refer to in Java?

- a) Refers to the parent class
- b) Refers to the current object of the class
- c) Refers to an object of the same class
- d) Refers to the class itself

Answer: b) Refers to the current object of the class

10. Which of the following is an example of method overloading in Java?

- a) Defining multiple methods with the same name but different parameter types
- b) Defining multiple methods with the same name and parameters
- c) Defining multiple methods with different names
- d) Changing the return type of a method

Answer: a) Defining multiple methods with the same name but different parameter types

11. What does constructor chaining mean in Java?

- a) Calling one constructor from another constructor within the same class

- b) Calling one constructor from a parent class constructor
- c) Chaining together multiple class constructors to form a class
- d) None of the above

Answer: a) Calling one constructor from another constructor within the same class

12. Which of the following can be inherited in Java?

- a) Methods
- b) Variables
- c) Constructors
- d) None of the above

Answer: a) Methods

13. What is the benefit of interface inheritance in Java?

- a) It allows classes to inherit the implementation of methods
- b) It allows multiple classes to share the same implementation

- c) It allows classes to inherit only method signatures without implementation
- d) None of the above

Answer: c) It allows classes to inherit only method signatures without implementation

14. Which of the following best describes overriding in Java?

- a) A subclass provides a specific implementation for a method already provided by its superclass
- b) A subclass provides a new method signature in the superclass
- c) A superclass method is hidden by another superclass method
- d) None of the above

Answer: a) A subclass provides a specific implementation for a method already provided by its superclass

15. Which of the following is true about final in Java?

- a) A final class cannot be extended
- b) A final method cannot be overridden
- c) A final variable's value cannot be changed

- d) All of the above

Answer: d) All of the above

16. Which of the following is an example of multilevel inheritance in Java?

- a) Class A inherits Class B which inherits Class C
- b) Class A inherits Class B and Class C
- c) Class A inherits Class B and implements Interface C
- d) Class A extends Class B and Interface C

Answer: a) Class A inherits Class B which inherits Class C

17. What is the keyword used to indicate a parent class constructor in Java?

- a) super()
- b) this()
- c) parent()
- d) base()

Answer: a) super()

18. Which of the following is an example of aggregation in Java?

- a) A department contains employees
- b) A car has a steering wheel
- c) A person has a car
- d) A person contains an address

Answer: c) A person has a car

19. What is dynamic method dispatch in Java?

- a) Determining which method to call at runtime
- b) Calling a method using reflection
- c) Calling a method through a variable
- d) None of the above

Answer: a) Determining which method to call at runtime

20. Which of the following is the result of calling `instanceof` in Java?

- a) It checks whether an object is of a specific class type

- b) It checks whether an object is an instance of the parent class
- c) It can only be used with interfaces
- d) All of the above

Answer: a) It checks whether an object is of a specific class type

3. Exception Handling & File Handling (20 MCQs)

1. Which of the following is true about exceptions in Java?

- a) An exception is an event that disrupts the normal flow of the program
- b) An exception is used to handle errors in the program
- c) An exception is a way to throw an error to the user
- d) All of the above

Answer: a) An exception is an event that disrupts the normal flow of the program

2. Which class is the base class for all exceptions in Java?

- a) Throwable
- b) Error
- c) Exception
- d) RuntimeException

Answer: a) Throwable

3. Which of the following exceptions can be handled by a program?

- a) Checked exceptions
- b) Unchecked exceptions
- c) Errors
- d) None of the above

Answer: a) Checked exceptions

4. What is the correct way to catch an exception in Java?

- a) try-catch
- b) catch-throw
- c) throw-catch
- d) try-throw

Answer: a) `try-catch`

5. Which of the following is used to manually throw an exception in Java?

- a) `throw`
- b) `throws`
- c) `exception`
- d) `catch`

Answer: a) `throw`

6. What is the purpose of the `finally` block in Java?

- a) It is used to execute code whether or not an exception occurs
- b) It is used to handle checked exceptions
- c) It is used to define an exception type
- d) It is used to throw an exception

Answer: a) It is used to execute code whether or not an exception occurs

7. Which of these exceptions are checked exceptions in Java?

- a) `ArithmeticException`
- b) `NullPointerException`
- c) `IOException`
- d) `ArrayIndexOutOfBoundsException`

Answer: c) `IOException`

8. What is the default behavior when an exception occurs in a `try` block and there is no `catch` block to handle it?

- a) The program terminates
- b) The exception is ignored
- c) The program continues executing the rest of the code
- d) The `finally` block is executed

Answer: a) The program terminates

9. What type of exception is thrown when you try to access an index outside the bounds of an array?

- a) `ArithmeticException`
- b) `NullPointerException`
- c) `ArrayIndexOutOfBoundsException`

- d) `ClassCastException`

Answer: c) `ArrayIndexOutOfBoundsException`

10. What does the `throws` keyword do in Java?

- a) It is used to throw an exception in a method
- b) It is used to declare an exception that a method can throw
- c) It is used to catch exceptions thrown by the method
- d) It defines a method that handles all exceptions

Answer: b) It is used to declare an exception that a method can throw

11. Which of these classes is used for reading from a file in Java?

- a) `FileWriter`
- b) `BufferedReader`
- c) `PrintWriter`
- d) `FileInputStream`

Answer: b) `BufferedReader`

12. Which of these methods is used to read a single character from a file in Java?

- a) `readLine()`
- b) `readChar()`
- c) `read()`
- d) `getChar()`

Answer: c) `read()`

13. Which exception is thrown when an attempt to read a file that does not exist is made?

- a) `FileNotFoundException`
- b) `IOException`
- c) `FileReadException`
- d) `NullPointerException`

Answer: a) `FileNotFoundException`

14. Which of the following is used to write to a file in Java?

- a) `FileReader`
- b) `FileWriter`

- c) `BufferedReader`
- d) `Scanner`

Answer: b) `FileWriter`

15. Which method is used to close a file in Java after reading/writing?

- a) `close()`
- b) `exit()`
- c) `shutdown()`
- d) `end()`

Answer: a) `close()`

16. Which of these classes is used for writing characters to a file in Java?

- a) `FileReader`
- b) `FileWriter`
- c) `BufferedWriter`
- d) `PrintWriter`

Answer: b) `FileWriter`

17. What is the purpose of the `File` class in Java?

- a) It is used to create new files and directories
- b) It is used for file reading and writing
- c) It is used to handle I/O exceptions
- d) It is used for error handling during file operations

Answer: a) It is used to create new files and directories

18. How do you check if a file exists using Java?

- a) `file.exists()`
- b) `exists(file)`
- c) `isFile(file)`
- d) `checkExists()`

Answer: a) `file.exists()`

19. Which of the following classes should be used for buffered file reading in Java?

- a) `BufferedReader`
- b) `FileReader`

- c) InputStreamReader
- d) FileWriter

Answer: a) BufferedReader

20. What is the correct syntax to read a file using **BufferedReader**?

**BufferedReader reader = new BufferedReader(new
FileReader("file.txt"));**

- a) **BufferedReader reader = new BufferedReader(new
FileReader("file.txt"));**
- b) **BufferedReader reader = new
FileReader("file.txt");**
- c) **BufferedReader reader = new
BufferedReader(file.txt);**
- d) **BufferedReader reader = new
FileReader("file.txt");**

Answer: a) **BufferedReader reader = new BufferedReader(new
FileReader("file.txt"));**

4. Multithreading & Collections (20 MCQs)

1. What is the main benefit of multithreading in Java?

- a) To execute multiple tasks simultaneously
- b) To increase CPU utilization
- c) To reduce memory usage
- d) Both a and b

Answer: d) Both a and b

2. What is a thread in Java?

- a) A single process
- b) A lightweight process
- c) A class
- d) A method

Answer: b) A lightweight process

3. Which interface should be implemented for creating a thread in Java?

- a) Thread
- b) Runnable
- c) Callable
- d) Executor

Answer: b) Runnable

4. How do you create a thread using the Runnable interface?

- a) By extending Thread class
- b) By implementing Runnable interface and passing it to a Thread object
- c) By creating a Thread object
- d) By calling Thread.run()

Answer: b) By implementing Runnable interface and passing it to a Thread object

5. Which method is used to start a thread in Java?

- a) run()
- b) start()
- c) execute()
- d) begin()

Answer: b) start()

6. What is the state of a thread after it has been started but before it finishes execution?

- a) New
- b) Runnable
- c) Blocked
- d) Terminated

Answer: b) Runnable

7. What happens if a thread calls the `sleep()` method in Java?

- a) The thread stops and terminates
- b) The thread pauses for the specified time
- c) The thread enters into a waiting state indefinitely
- d) The thread is blocked by the CPU

Answer: b) The thread pauses for the specified time

8. Which of the following is true about synchronized methods in Java?

- a) Only one thread can execute a synchronized method at a time
- b) All threads can execute synchronized methods concurrently

- c) Synchronized methods are faster than non-synchronized methods
- d) None of the above

Answer: a) Only one thread can execute a synchronized method at a time

9. What is a deadlock in Java?

- a) When two threads can never execute together
- b) When two threads block each other by holding locks
- c) When a thread stops executing unexpectedly
- d) When a thread is blocked by the CPU

Answer: b) When two threads block each other by holding locks

10. How can you prevent a deadlock in Java?

- a) By using **synchronized** methods
- b) By using **wait()** and **notify()**
- c) By acquiring locks in a predefined order
- d) By using **Thread.sleep()** in between

Answer: c) By acquiring locks in a predefined order

11. What is the purpose of the `ExecutorService` in Java?

- a) To manage the lifecycle of threads
- b) To execute tasks asynchronously
- c) To handle thread synchronization
- d) To communicate between threads

Answer: b) To execute tasks asynchronously

12. What is the difference between `ArrayList` and `LinkedList` in Java?

- a) `ArrayList` is faster than `LinkedList` for adding/removing elements
- b) `LinkedList` stores elements as nodes, while `ArrayList` stores elements as arrays
- c) `ArrayList` allows random access, while `LinkedList` does not
- d) Both a and b

Answer: d) Both a and b

13. Which of the following classes implements the `List` interface in Java?

- a) HashSet
- b) TreeMap
- c) LinkedList
- d) PriorityQueue

Answer: c) LinkedList

14. Which of these is an implementation of the **Set** interface in Java?

- a) ArrayList
- b) HashSet
- c) TreeMap
- d) HashMap

Answer: b) HashSet

15. What does the **contains()** method in the **Set** interface check?

- a) If the set is empty
- b) If the set contains the specified element
- c) If the set has duplicate elements

- d) None of the above

Answer: b) If the set contains the specified element

16. Which of the following methods is used to add elements in a **Map**?

- a) **put()**
- b) **add()**
- c) **insert()**
- d) **set()**

Answer: a) **put()**

17. What is the function of the **Iterator** interface in Java?

- a) It allows iteration over elements in a collection
- b) It is used to iterate over an array
- c) It provides functionality to add and remove elements
- d) It is used to sort the elements of a collection

Answer: a) It allows iteration over elements in a collection

18. Which of the following collections guarantees the order of elements?

- a) HashSet
- b) TreeSet
- c) LinkedHashMap
- d) PriorityQueue

Answer: c) LinkedHashMap

19. What does the `compareTo()` method do in the `Comparable` interface?

- a) Compares two objects based on some natural order
- b) Compares two objects' references
- c) Compares two objects' hash codes
- d) None of the above

Answer: a) Compares two objects based on some natural order

20. What type of collection is `HashMap` in Java?

- a) A collection of unique elements
- b) A collection of key-value pairs

- c) A collection that maintains insertion order
- d) A collection that implements **Set** interface

Answer: b) A collection of key-value pairs

5. Advanced Java Topics (20 MCQs)

1. What is the main advantage of using Java Reflection?

- a) It allows runtime inspection and modification of classes
- b) It allows better performance in multithreaded applications
- c) It is used for network communication
- d) It reduces the size of the Java program

Answer: a) It allows runtime inspection and modification of classes

2. Which of the following is used for Object Serialization in Java?

- a) `Serializable` interface
- b) `Externalizable` interface
- c) Both a and b
- d) `Cloneable` interface

Answer: c) Both a and b

3. What is the difference between `Serializable` and `Externalizable` interfaces in Java?

- a) `Serializable` provides default serialization, while `Externalizable` gives full control over serialization
- b) `Serializable` requires a no-argument constructor
- c) `Externalizable` provides default serialization, while `Serializable` gives full control
- d) Both are used for deep cloning of objects

Answer: a) `Serializable` provides default serialization, while `Externalizable` gives full control over serialization

4. What is the main purpose of the `transient` keyword in Java?

- a) To prevent a variable from being serialized

- b) To allow a variable to be serialized
- c) To mark a method as not being able to be overridden
- d) To prevent inheritance

Answer: a) To prevent a variable from being serialized

5. What is the role of `java.nio` package in Java?

- a) To provide a more scalable and flexible I/O model
- b) To implement security features in Java
- c) To work with threads and concurrency
- d) To manage Java database connectivity

Answer: a) To provide a more scalable and flexible I/O model

6. Which class is used to handle input and output in Java NIO?

- a) `FileInputStream`
- b) `BufferedReader`
- c) `FileChannel`
- d) `InputStreamReader`

Answer: c) `FileChannel`

7. What is the difference between `wait()` and `sleep()` methods in Java?

- a) `wait()` is used for inter-thread communication, while `sleep()` pauses a thread for a specific time
- b) `sleep()` is used for inter-thread communication, while `wait()` pauses a thread
- c) Both methods pause the thread indefinitely
- d) Both methods are used to start a thread

Answer: a) `wait()` is used for inter-thread communication, while `sleep()` pauses a thread for a specific time

8. What is the purpose of the `volatile` keyword in Java?

- a) To mark a variable that is shared by multiple threads
- b) To prevent a thread from executing
- c) To lock a resource for synchronization
- d) To force a thread to terminate

Answer: a) To mark a variable that is shared by multiple threads

9. Which of the following is used to implement a singleton pattern in Java?

- a) A constructor with the `private` modifier
- b) A static method to return the single instance
- c) Both a and b
- d) A static initializer block

Answer: c) Both a and b

10. What is the main function of Java's `java.util.concurrent` package?

- a) To provide classes and interfaces for concurrency utilities
- b) To manage database connections
- c) To handle file I/O
- d) To create GUI components

Answer: a) To provide classes and interfaces for concurrency utilities

11. What is the significance of `ThreadLocal` in Java?

- a) It provides thread-specific storage for variables

- b) It guarantees a single thread will access the variable
- c) It makes a variable accessible to all threads
- d) It ensures thread synchronization

Answer: a) It provides thread-specific storage for variables

12. Which of these is true about Java's **Callable** interface?

- a) It is used to return a result from a thread
- b) It is used to handle exceptions in threads
- c) It is similar to **Runnable** but with additional functionality
- d) All of the above

Answer: d) All of the above

13. Which of the following is a feature of the Java **Fork/Join** framework?

- a) It is used to parallelize CPU-intensive tasks
- b) It divides a task into smaller sub-tasks
- c) It merges the results of sub-tasks

- d) All of the above

Answer: d) All of the above

14. What is the use of `java.lang.invoke` package in Java?

- a) To provide low-level APIs for working with method handles and `invokedynamic`
- b) To manage database connections
- c) To implement custom class loaders
- d) To perform file input and output

Answer: a) To provide low-level APIs for working with method handles and `invokedynamic`

15. How does Java's `independent` garbage collection differ from traditional garbage collection?

- a) It is managed by multiple threads
- b) It is triggered manually by the developer
- c) It uses multiple algorithms for different types of memory regions
- d) None of the above

Answer: c) It uses multiple algorithms for different types of memory regions

16. What is Java's **stream** API primarily used for?

- a) To provide an easier way to manipulate collections of data
- b) To optimize file handling operations
- c) To handle concurrency issues in multithreading
- d) To connect to remote servers

Answer: a) To provide an easier way to manipulate collections of data

17. What does the **default** keyword in Java interfaces define?

- a) A method that can be implemented by classes inheriting the interface
- b) A default implementation of a method in an interface
- c) A default instance variable in an interface
- d) None of the above

Answer: b) A default implementation of a method in an interface

18. What does the `@FunctionalInterface` annotation indicate in Java?

- a) That the interface can have multiple abstract methods
- b) That the interface is designed to support lambda expressions
- c) That the interface can have static methods
- d) That the interface supports default methods

Answer: b) That the interface is designed to support lambda expressions

19. What is the purpose of the `@Override` annotation in Java?

- a) It indicates that a method is overriding a superclass method
- b) It marks a method as deprecated
- c) It indicates a method is private
- d) None of the above

Answer: a) It indicates that a method is overriding a superclass method

20. What is a `WeakReference` in Java?

- a) A reference that prevents garbage collection of the referred object
- b) A reference that allows an object to be collected by the garbage collector
- c) A reference that automatically cleans up objects
- d) A reference that is used for multithreading

Answer: b) A reference that allows an object to be collected by the garbage collector

Pluto Academy

30 Basic Java Interview Questions

1. What is the difference between JDK, JRE, and JVM?
2. Explain the difference between `==` and `.equals()` in Java.
3. What are wrapper classes in Java?
4. Why is Java platform-independent?
5. What is the difference between `final`, `finally`, and `finalize` in Java?
6. How does Java handle memory management?
7. What are the access modifiers in Java?
8. What is the difference between `static` and instance methods?
9. What is autoboxing and unboxing in Java?
10. What is the difference between `String`, `StringBuilder`, and `StringBuffer`?
11. What is a constructor? How is it different from a method?
12. What is the difference between `break` and `continue` statements?
13. Can we overload the `main` method in Java?

14. What is type casting in Java? Explain upcasting and downcasting.
15. Explain the differences between checked and unchecked exceptions.
16. What is the difference between `ArrayList` and `LinkedList`?
17. What is method overloading and method overriding?
18. What is the purpose of the `super` keyword in Java?
19. What is the difference between `throw` and `throws`?
20. Explain the concept of pass-by-value in Java.
21. What is the difference between a shallow copy and a deep copy?
22. What is a `static` block in Java?
23. What is an interface in Java, and how is it different from an abstract class?
24. What are default methods in interfaces?
25. How does `hashCode()` and `equals()` work in Java?
26. What is the use of the `transient` keyword in Java?
27. What is the difference between `volatile` and `synchronized`?
28. What is `instanceof` in Java?

- 29. What is a `lambda expression` in Java?
- 30. What is the difference between `Comparator` and `Comparable`?

30 OOP & Data Structures Interview Questions

- 31. What are the four pillars of Object-Oriented Programming (OOP)?
- 32. How does inheritance work in Java?
- 33. What is polymorphism in Java? Explain with an example.
- 34. What is encapsulation? Why is it important?
- 35. What is abstraction? How is it different from encapsulation?
- 36. Can we override a private method in Java?
- 37. What is the purpose of an abstract class in Java?
- 38. What is multiple inheritance? Does Java support it?
- 39. Can we create an object of an abstract class?
- 40. How do you prevent method overriding in Java?
- 41. What is an inner class in Java?
- 42. How does method resolution work in Java (dynamic method dispatch)?
- 43. What is the Liskov Substitution Principle (LSP) in OOP?
- 44. What is the Open-Closed Principle in Java?

45. How is object cloning implemented in Java?
46. What is the difference between deep and shallow copy in Java?
47. What are the different types of design patterns in Java?
48. Explain the Singleton Design Pattern.
49. What is the Factory Design Pattern?
50. What is the Observer Design Pattern?
51. How does a HashMap work internally in Java?
52. What is the difference between `HashMap` and `HashTable`?
53. What are the differences between `ArrayList` and `Vector`?
54. What is a `LinkedList`, and how does it differ from an `ArrayList`?
55. What is a `Stack`, and how is it implemented in Java?
56. What is the time complexity of searching in a `HashMap`?
57. What is a binary search tree (BST)?
58. What is a graph, and how can it be implemented in Java?
59. What is the difference between BFS and DFS?
60. How do you reverse a `LinkedList` in Java?

20 Advanced Java & Multithreading Questions

61. What is multithreading in Java?
62. What is the difference between a thread and a process?
63. How do you create a thread in Java?
64. What is the difference between `Runnable` and `Thread`?
65. What is a daemon thread in Java?
66. How do you prevent thread interference in Java?
67. What is the `synchronized` keyword in Java?
68. What is a deadlock? How can it be avoided?
69. What are `wait()`, `notify()`, and `notifyAll()` methods?
70. What is a thread pool? How does it improve performance?
71. What is the difference between `Callable` and `Runnable`?
72. How does the `ReentrantLock` work in Java?
73. What is the `ForkJoinPool` framework?
74. What is the difference between parallelism and concurrency?
75. What are atomic variables in Java?
76. How does the `CompletableFuture` class work?
77. What is the difference between `CountDownLatch` and `CyclicBarrier`?

- 78. What is the difference between parallel streams and sequential streams in Java?
- 79. What is the Executor framework in Java?
- 80. How do you handle race conditions in Java?

20 Problem-Solving & Logical Thinking Questions

- 81. How do you find the first non-repeating character in a string?
- 82. Write a Java program to check if a number is prime.
- 83. How do you find the largest and smallest number in an unsorted array?
- 84. How do you reverse a string without using the `reverse()` method?
- 85. Write a Java program to check if a given string is a palindrome.
- 86. How do you implement a custom sorting order in Java?
- 87. How do you remove duplicates from an array in Java?
- 88. Write a Java program to find the missing number in an array of 1 to N.
- 89. How do you find the longest substring without repeating characters?
- 90. How do you detect a loop in a LinkedList?
- 91. Write a Java program to find the intersection of two arrays.

92. How do you implement a binary search algorithm in Java?
93. How do you implement a LRU (Least Recently Used) Cache in Java?
94. Write a program to check if a given binary tree is balanced.
95. How do you implement a stack using a queue?
96. How do you find the kth largest element in an array?
97. How do you find the median of two sorted arrays?
98. How do you check if a given string is an anagram of another string?
99. How do you find the duplicate number in an array of N integers?
100. Write a Java program to merge two sorted linked lists.

Congratulations! 🎉 You have successfully journeyed through Java from basics to advanced concepts, covering everything from syntax fundamentals to real-world applications.

- By Pluto Academy

Pluto Academy